

ENTERPRISE EDUCATION EDITION

Playwright Academy Handbook

A complete curriculum of engineering principles, code designs,
and automation blueprints.



PLAYWRIGHT CURRICULUM ENGINEERING BOARD © 2026

PHASE 0 – PREREQUISITES

CHAPTER 01

Your Computer's Command Line

 15 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Open a terminal on Windows, Mac, or Linux
- ✓ Navigate folders using cd commands
- ✓ Create files and folders from the command line

CHAPTER 1 • Your Computer's Command Line

1. Why This Matters

Every tool you will use — Node.js, npm, Playwright — is controlled from the command line. If you cannot open a terminal and navigate folders, you cannot run a single test.

2. Conceptual Overview

A terminal (also called a command line, shell, or console) is a text-based interface to your computer. Instead of clicking icons, you type commands.

On Windows, you will use PowerShell. On Mac or Linux, you will use Terminal.

Here are the essential commands you need to know:

Opening a terminal:

Windows: Press the Windows key, type "PowerShell", and click it.

Mac: Press Cmd + Space, type "Terminal", and press Enter.

Navigating folders:

`cd foldername` — moves into a folder.

`cd ..` — moves back up one level.

`dir` (Windows) or `ls` (Mac/Linux) — lists files in the current folder.

Creating things:

`mkdir my-project` — creates a new folder called "my-project".

`New-Item hello.txt` (Windows) or `touch hello.txt` (Mac/Linux) — creates an empty file.

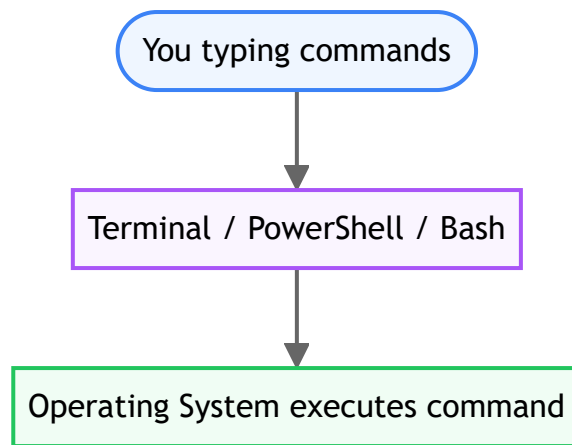
Understanding paths:

When you open a terminal, you start in a "home" folder.

Every file on your computer has an address called a path (like `C:\Users\you\Desktop\file.txt`).

The terminal always shows your current location. This is called the "working directory".

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
# Step 1: Open your terminal (PowerShell on Windows)

# Step 2: See where you are right now
cd ~

# Step 3: Create a project folder
mkdir playwright-learning

# Step 4: Move into that folder
cd playwright-learning

# Step 5: Verify you are inside the folder
pwd
# Output: C:\Users\YourName\playwright-learning (Windows)
# Output: /Users/YourName/playwright-learning (Mac)

# Step 6: List contents (it will be empty)
dir    # Windows
ls     # Mac/Linux
```

4. Best Practices (Do's and Don'ts)

 Do

Practice navigating folders every day until `cd` and `dir/ls` feel natural.

Always check your current directory with `pwd` before running commands.

● Don't

Do not copy-paste commands blindly — type them by hand to build muscle memory.

Do not panic if you see an error. Read the message carefully — it usually tells you what went wrong.

5. Chapter Summary

The terminal is a text-based way to control your computer.

Use `cd` to move between folders, `dir/ls` to list files, and `mkdir` to create folders.

Always know your current working directory before running commands.

6. Exercises & Mini-Project

Exercises

Create a folder called "test-project" inside your home directory, navigate into it, and then navigate back out.

▼ Exercise 1 Solution

```
code CODE
# Exercise 1 Solution:
mkdir test-project
cd test-project
pwd          # Verify you are inside test-project
cd ..
pwd          # Verify you are back in the parent folder
```

List all files on your Desktop folder using only terminal commands.

▼ 💡 Exercise 2 Solution

```
code CODE
# Exercise 2 Solution (Windows):
cd ~\Desktop
dir

# Exercise 2 Solution (Mac/Linux):
cd ~/Desktop
ls
```

🚀 Mini-Project

Create the following folder structure using only terminal commands: playwright-learning/tests/, playwright-learning/pages/, playwright-learning/data/.

▼ 💡 Mini-Project Solution

```
code CODE
# Mini-Project Solution:
mkdir playwright-learning
cd playwright-learning
mkdir tests
mkdir pages
mkdir data
dir    # Verify all three folders exist
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Open your terminal and create the project folder that will hold your entire TodoMVC automation project:

code

CODE

```
mkdir todomvc-automation
cd todomvc-automation
pwd
```

This folder will be your home base for the next 32 chapters.

8. Interview Q&A Preparation

Q1: Why is the command line important for test automation?

Expected Answer: All automation tools (Node.js, npm, Playwright) are controlled via CLI commands. CI/CD servers do not have graphical interfaces — everything runs through terminal commands.

9. Chapter Cheat Sheet

code

CODE

```
cd folder      # Enter folder\ncd ..           # Go back\ndir / ls       # List
```

✓ Completed Chapter 01 / 40

3%

PHASE 0 – PREREQUISITES

PHASE 0 – PREREQUISITES

CHAPTER 02

Installing Node.js & npm

 10 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Download and install Node.js
- ✓ Verify Node.js and npm are working
- ✓ Understand what npm does

CHAPTER 2 • Installing Node.js & npm

1. Why This Matters

Node.js is the engine that runs JavaScript outside the browser. npm is the package manager that installs tools like Playwright. Without these two, nothing works.

2. Conceptual Overview

Node.js is a program that lets you run JavaScript on your computer (not just in a web browser). npm (Node Package Manager) comes bundled with Node.js and is the tool that downloads and installs libraries and frameworks.

Installation steps:

Go to <https://nodejs.org>

Download the LTS (Long Term Support) version — it is the stable, recommended version.

Run the installer. Accept all default options. Click "Next" through every screen.

When installation completes, close and reopen your terminal (this is important!).

Verify the installation by typing two commands.

What is npm?

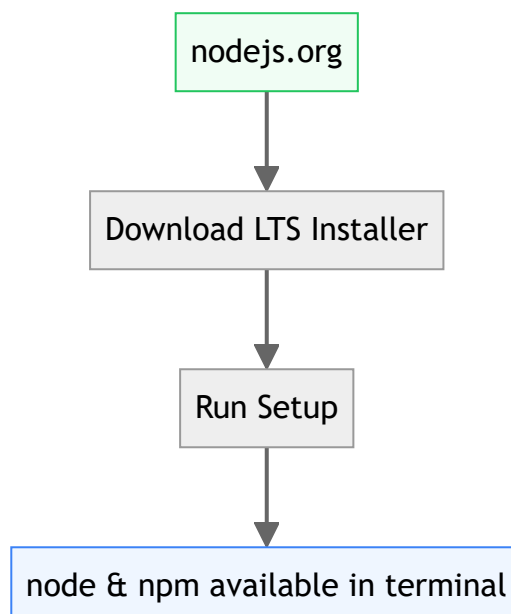
npm is like an app store for JavaScript code.

When you type `npm install playwright`, npm downloads Playwright from the internet and saves it in your project folder.

The downloaded packages go into a folder called `node_modules`.

The list of what you installed is saved in a file called `package.json`.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
# After installing Node.js, open a NEW terminal window and type:

node -v
# Expected output: v20.x.x (or similar version number)

npm -v
# Expected output: 10.x.x (or similar version number)

# If both commands show version numbers, you are ready!
# If you see "command not found", close the terminal, reopen it, and try again.
```

4. Best Practices (Do's and Don'ts)

Do

Always download the LTS version from nodejs.org.

Reopen your terminal after installing Node.js.

Don't

Do not install the "Current" version — it may have unstable features.

Do not skip the verification step (`node -v` and `npm -v`).

5. Chapter Summary

Node.js lets you run JavaScript on your computer.

npm downloads and installs packages like Playwright.

Always verify installation with `node -v` and `npm -v`.

6. Exercises & Mini-Project

Exercises

Install Node.js and verify the version numbers appear in your terminal.

▼ 💡 Exercise 1 Solution

```
code CODE  
# Exercise 1 Solution:  
node -v  
# Output: v20.15.0 (your version may differ)  
npm -v  
# Output: 10.7.0 (your version may differ)
```

Run `node -e "console.log(2 + 3)"` in your terminal and observe the output.

▼ 💡 Exercise 2 Solution

```
code CODE  
# Exercise 2 Solution:  
node -e "console.log(2 + 3)"  
# Output: 5  
# This proves Node.js can execute JavaScript from your terminal!
```

🚀 Mini-Project

Create a new folder, navigate into it, and run `npm init -y` to generate a `package.json` file. Open the file and read its contents.

▼ 💡 Mini-Project Solution

```
code CODE  
# Mini-Project Solution:  
mkdir my-first-project  
cd my-first-project  
npm init -y  
# Output: Wrote to ../package.json  
# Open the file to see: { "name": "my-first-project", "version": "1.0
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Navigate into your `todomvc-automation` folder and initialize it as a Node.js project:

```
code CODE  
cd todomvc-automation  
npm init -y
```

This creates `package.json` — the file that tracks your project dependencies.

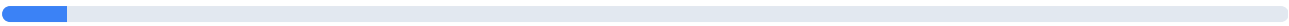
🎯 8. Interview Q&A Preparation

Q1: What is the difference between Node.js and npm?

Expected Answer: Node.js is the JavaScript runtime engine. npm is the package manager that comes with Node.js, used to install third-party libraries and tools.

📄 9. Chapter Cheat Sheet

```
code CODE  
node -v          # Check Node version\nnpm -v           # Check npm version
```

✓ Completed Chapter 02 / 40

5%

PHASE 0 – PREREQUISITES

PHASE 0 - PREREQUISITES

CHAPTER 03

Your First JavaScript Program

 10 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Create a JavaScript file
- ✓ Write and run `console.log` statements
- ✓ Understand the write-run-observe cycle

CHAPTER 3 • Your First JavaScript Program

1. Why This Matters

Before you can write a test, you need to know how to create a file, write code in it, and run it. This chapter gives you that fundamental feedback loop.

2. Conceptual Overview

Programming is a cycle: you write instructions in a file, run the file, observe the result, and then improve. This chapter teaches that cycle using the simplest possible program.

What is `console.log`?

It is a command that prints text to your terminal.

Think of it as the "print" button for code.

It is the most important debugging tool you will ever use.

How to create a file:

Open your terminal in your project folder.

You can create a file using the terminal or a text editor.

We recommend installing VS Code (<https://code.visualstudio.com>) — it is a free text editor built for coding.

In VS Code, open your project folder (File → Open Folder), then create a new file.

Your first program:

Create a file called `hello.js` (the `.js` extension means JavaScript).

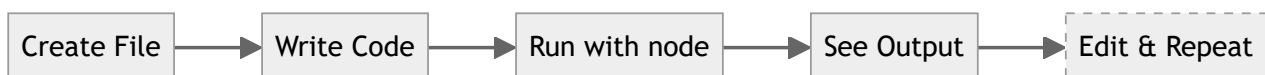
Type `console.log("Hello, World!");` inside the file.

Save the file.

In your terminal, type `node hello.js` and press Enter.

You will see "Hello, World!" printed in the terminal.

Execution Flow Diagram



3. Implementation & Code Examples

hello.js

TS

```
// This is your very first program!

console.log("Hello, World!");
console.log("My name is [Your Name]");
console.log("I am learning Playwright!");
console.log("2 + 3 =", 2 + 3);

// To run this file, type in your terminal:
// node hello.js

// Expected output:
// Hello, World!
// My name is [Your Name]
// I am learning Playwright!
// 2 + 3 = 5
```

4. Best Practices (Do's and Don'ts)

Do

Install VS Code and use it to write all your code.

Save your file before running it (Ctrl+S / Cmd+S).

Read the output after every run — it tells you exactly what happened.

Don't

Do not forget the semicolon at the end of each statement (it is optional in JS but good practice).

Do not forget to save the file before running node hello.js — you will run the old version.

5. Chapter Summary

Programming is a write → run → observe → edit cycle.

console.log() prints output to your terminal.

Create .js files and run them with the node command.

6. Exercises & Mini-Project

Exercises

Create a file called math.js that prints the results of $10 * 5$, $100 / 4$, and $7 - 3$.

▼ 💡 Exercise 1 Solution

```
code CODE  
// Exercise 1 Solution – math.js  
console.log("10 * 5 =", 10 * 5); // 50  
console.log("100 / 4 =", 100 / 4); // 25  
console.log("7 - 3 =", 7 - 3); // 4
```

Create a file called about-me.js that prints your name, age, and favorite food on separate lines.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution – about-me.js  
console.log("Name: Alice");  
console.log("Age: 28");  
console.log("Favorite food: Pizza");
```

🚀 Mini-Project

Build a file called calculator.js that calculates and prints the total cost of 3 items: a book (\$12.99), a pen (\$2.50), and a notebook (\$5.75). Print each item and the total.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution – calculator.js
const book = 12.99;
const pen = 2.50;
const notebook = 5.75;

console.log("Book: $" + book);
console.log("Pen: $" + pen);
console.log("Notebook: $" + notebook);

const total = book + pen + notebook;
console.log("Total: $" + total);
// Output: Total: $21.24
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Inside your `todomvc-automation` folder, create a file called `plan.js` and run it:

```
plan.js JS

console.log("≡ TodoMVC Automation Project ≡");
console.log("Target: https://demo.playwright.dev/todomvc");
console.log("Goal: Full E2E test suite with Page Objects and CI/CD");
console.log("Starting from zero. Let's go!");
```

Run: `node plan.js`

🎯 8. Interview Q&A Preparation

Q1: What does `console.log` do?

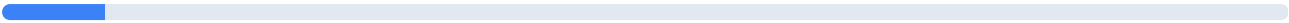
Expected Answer: It prints output to the terminal. It is the primary tool for debugging and verifying that code is executing correctly.

📄 9. Chapter Cheat Sheet

code

CODE

```
console.log("text");    // Print text\nnode filename.js        // Run a JS
```


✓ Completed Chapter 03 / 40

8%

PHASE 0 – PREREQUISITES

PHASE 0 – PREREQUISITES

CHAPTER 04

Variables, Conditions & Loops

 20 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Store values in variables
- ✓ Make decisions with if/else
- ✓ Repeat actions with for loops

CHAPTER 4 • Variables, Conditions & Loops

1. Why This Matters

Every test stores values (URLs, usernames), makes decisions (did the button appear?), and repeats actions (check every row in a table). These are the three pillars of programming.

2. Conceptual Overview

This chapter teaches the three building blocks that every program uses.

Variables — Storing Data:

A variable is a named container that holds a value.

`const name = "Alice";` — creates a constant (cannot be changed).

`let age = 25;` — creates a variable that can be updated.

You can store text (strings), numbers, true/false (booleans), and more.

Conditions — Making Decisions:

`if` checks whether something is true, and runs code accordingly.

`else` runs when the condition is false.

`else if` adds additional checks.

Comparison operators: `===` (equals), `!==` (not equals), `>` (greater than), `<` (less than).

Loops — Repeating Actions:

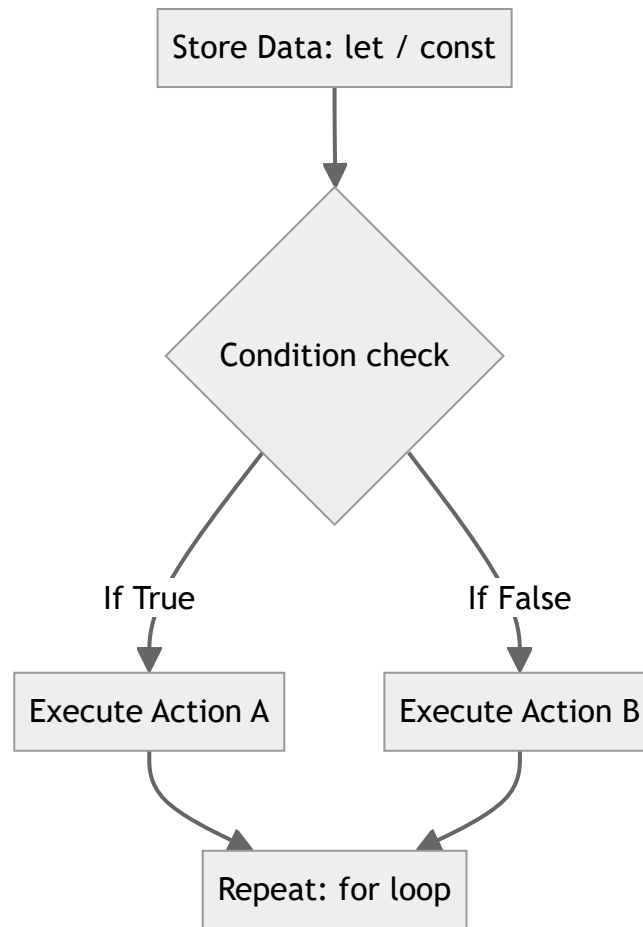
A for loop repeats a block of code a specific number of times.

It has three parts: start (let `i = 0`), condition (`i < 5`), and step (`i++`).

`i++` means "add 1 to `i` after each loop".

Loops are essential for checking every item in a list or every row in a table.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// 1. VARIABLES – Storing data
const websiteUrl = "https://demo.playwright.dev/todomvc";
let todoCount = 0;
const isLoggedIn = false;

console.log("Website:", websiteUrl);
console.log("Todos:", todoCount);
console.log("Logged in:", isLoggedIn);

// 2. CONDITIONS – Making decisions
const statusCode = 404;

if (statusCode === 200) {
  console.log("Success! Page loaded.");
} else if (statusCode === 404) {
  console.log("Error! Page not found.");
} else {
  console.log("Unknown status:", statusCode);
}

// 3. LOOPS – Repeating actions
console.log("Checking 5 table rows:");
for (let i = 1; i ≤ 5; i++) {
  console.log("Checking row", i);
}
```

4. Best Practices (Do's and Don'ts)

Do

Use `const` for values that never change (URLs, config) and `let` for values that do (counters, flags).

Always use `===` (triple equals) for comparisons, not `==` (double equals).

Read loop code from left to right: start → condition → step.

Don't

Do not use `var` — it causes scope bugs (you will learn why in Chapter 1).

Do not create infinite loops — always ensure the loop condition will eventually be false.

5. Chapter Summary

Variables store data: `const` for fixed values, `let` for changing values.

Conditions (`if/else`) let code make decisions based on runtime states.

Loops (`for`) repeat actions a specific number of times.

6. Exercises & Mini-Project

Exercises

Write a script that stores a test score in a variable and prints "PASS" if it is 70 or above, and "FAIL" otherwise.

Exercise 1 Solution

```
code CODE  
  
// Exercise 1 Solution:  
const testScore = 85;  
  
if (testScore >= 70) {  
  console.log("PASS");  
} else {  
  console.log("FAIL");  
}
```

Write a loop that prints the numbers 1 through 10, and for each number prints whether it is even or odd.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
for (let i = 1; i <= 10; i++) {  
  if (i % 2 === 0) {  
    console.log(i, "is even");  
  } else {  
    console.log(i, "is odd");  
  }  
}
```

🚀 Mini-Project

Build a password strength checker: store a password string, check its length, and print "Weak" (under 6 chars), "Medium" (6-10 chars), or "Strong" (over 10 chars).

▼ 💡 Mini-Project Solution

```
code CODE  
// Mini-Project Solution – password-checker.js  
const password = "MySecretPass123";  
  
if (password.length < 6) {  
  console.log("Weak – password is too short (" + password.length + "  
} else if (password.length <= 10) {  
  console.log("Medium – password is okay (" + password.length + " cha  
} else {  
  console.log("Strong – password is solid (" + password.length + " ch  
}
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Create `todo-validator.js` that validates todo titles before they are added:

javascript

JS

```
const todoTitle = "Buy groceries";

if (todoTitle.length === 0) {
  console.log("Error: Todo title cannot be empty");
} else if (todoTitle.length > 100) {
  console.log("Error: Todo title is too long");
} else {
  console.log("Valid todo: " + todoTitle);
}

// Bonus: Loop through multiple todos
const todos = ["Buy groceries", "", "Read Playwright docs", "A very long todo :"];
for (let i = 0; i < todos.length; i++) {
  console.log("Checking todo " + (i + 1) + ": " + (todos[i].length === 0 ? "Empty" : "Valid"));
}
```

8. Interview Q&A Preparation

Q1: What is the difference between const and let?

Expected Answer: const creates a variable that cannot be reassigned. let creates a variable that can be updated. const is preferred for values that should not change.

9. Chapter Cheat Sheet

code

CODE

```
const x = 5;           // Constant\nlet y = 10;           // Mutable\
```

✓ Completed Chapter 04 / 40

10%

PHASE 0 – PREREQUISITES

PHASE 0 - PREREQUISITES

CHAPTER 05

Objects, Arrays & JSON

 20 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Create objects with key-value pairs
- ✓ Store lists of items in arrays
- ✓ Use array methods like map and filter

CHAPTER 5 • Objects, Arrays & JSON

1. Why This Matters

Test data (usernames, URLs, form inputs) is stored as objects. Table rows, dropdown options, and search results are arrays. You will use these data structures in every single test.

2. Conceptual Overview

Two data structures form the backbone of all JavaScript programs:

Objects — Named Key-Value Pairs:

An object groups related data under named keys.

Syntax: { key: value, key2: value2 }

Access values using dot notation: `user.name` or bracket notation: `user["name"]`.

Objects represent real things: a user, a product, a test configuration.

Arrays — Ordered Lists:

An array is a list of items in a specific order.

Syntax: [item1, item2, item3]

Access items by index (starting from 0): `fruits[0]` gives the first item.

Arrays represent collections: table rows, dropdown options, search results.

Useful Array Methods:

`.push(item)` — adds an item to the end.

`.length` — tells you how many items exist.

`.forEach(fn)` — runs a function on every item.

`.map(fn)` — transforms every item and returns a new array.

`.filter(fn)` — returns only items that pass a condition.

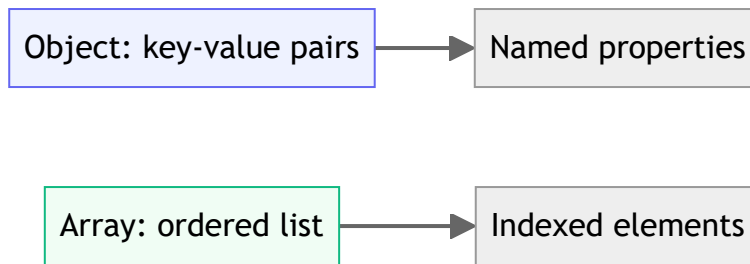
JSON (JavaScript Object Notation):

JSON is the universal data format for APIs and configuration files.

`JSON.stringify(obj)` converts an object to a text string.

`JSON.parse(str)` converts a text string back to an object.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// OBJECTS – Storing structured data
const user = {
  name: "Alice",
  email: "alice@test.com",
  age: 28,
  isAdmin: false
};

console.log("User:", user.name);      // Alice
console.log("Email:", user.email);    // alice@test.com

// ARRAYS – Storing lists
const todoItems = ["Buy groceries", "Write tests", "Read docs"];

console.log("First todo:", todoItems[0]); // Buy groceries
console.log("Total todos:", todoItems.length); // 3

// Loop through an array
todoItems.forEach((item, index) => {
  console.log((index + 1) + ".", item);
});

// Filter: get only items containing "test"
const testItems = todoItems.filter(item => item.toLowerCase().includes("test"));
console.log("Test items:", testItems); // ["Write tests"]

// Map: transform items to uppercase
const upperItems = todoItems.map(item => item.toUpperCase());
console.log("Upper:", upperItems);
```

4. Best Practices (Do's and Don'ts)

Do

Use objects to group related properties (user data, config settings).

Use arrays when you have a list of similar items.

Prefer `.forEach()`, `.map()`, and `.filter()` over manual for loops.

● Don't

Do not forget that array indexes start at 0, not 1.

Do not modify an array while looping through it — it causes skipped items.

5. Chapter Summary

Objects store named key-value pairs: { name: "Alice", age: 28 }.

Arrays store ordered lists: ["item1", "item2", "item3"].

Array methods (map, filter, forEach) process items without manual loops.

6. Exercises & Mini-Project

🎯 Exercises

Create an object representing a product (name, price, inStock) and print each property.

💡 Exercise 1 Solution

```
code CODE

// Exercise 1 Solution:
const product = {
  name: "Wireless Mouse",
  price: 29.99,
  inStock: true
};

console.log("Product:", product.name);
console.log("Price: $" + product.price);
console.log("In Stock:", product.inStock);
```

Create an array of 5 numbers and use `.filter()` to get only the numbers greater than 10.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
const numbers = [3, 15, 7, 22, 9];  
const bigNumbers = numbers.filter(n => n > 10);  
console.log("Numbers > 10:", bigNumbers); // [15, 22]
```

🚀 Mini-Project

Build a contact book: create an array of 3 user objects, each with name, email, and role. Print a formatted list, then filter to show only users with role "admin".

▼ 💡 Mini-Project Solution

```
code CODE  
// Mini-Project Solution – contact-book.js  
const contacts = [  
  { name: "Alice", email: "alice@test.com", role: "admin" },  
  { name: "Bob", email: "bob@test.com", role: "user" },  
  { name: "Carol", email: "carol@test.com", role: "admin" }  
];  
  
console.log("=== All Contacts ===");  
contacts.forEach((c, i) => {  
  console.log((i + 1) + ". " + c.name + " (" + c.role + ") - " + c.em  
});  
  
const admins = contacts.filter(c => c.role === "admin");  
console.log("\n=== Admins Only ===");  
admins.forEach(a => console.log("- " + a.name));
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Create `todo-data.js` that models todo items as objects in an array:

javascript

JS

```
const todos = [
  { title: "Buy groceries", completed: false },
  { title: "Write first test", completed: false },
  { title: "Install Node.js", completed: true }
];

const pending = todos.filter(t => !t.completed);
const done = todos.filter(t => t.completed);
console.log("Pending:", pending.length, "| Done:", done.length);
```

8. Interview Q&A Preparation

Q1: What is the difference between an object and an array?

Expected Answer: An object stores data with named keys (like a dictionary). An array stores data in an ordered list accessed by numeric index. Use objects for structured entities, arrays for collections.

9. Chapter Cheat Sheet

code

CODE

```
const obj = { key: "val" }; // Object\nconst arr = [1, 2, 3]; // A
```

✓ Completed Chapter 05 / 40

13%

PHASE 0 - PREREQUISITES

PHASE 0 – PREREQUISITES

CHAPTER 06

Functions & Importing Files

 15 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Write reusable functions
- ✓ Understand arrow function syntax
- ✓ Split code across multiple files using import/export

CHAPTER 6 • Functions & Importing Files

1. Why This Matters

Tests are built from reusable functions. Page Objects, helpers, and utilities are all functions organized in separate files. Understanding import/export is how you structure a real project.

2. Conceptual Overview

A function is a reusable block of code that performs a specific task. Instead of writing the same code over and over, you write it once inside a function and call it whenever you need it.

Basic Functions:

```
function greet(name) { return "Hello, " + name; }
```

Call it: `greet("Alice")` returns "Hello, Alice".

Parameters are inputs. Return values are outputs.

Arrow Functions:

A shorter way to write functions: `const greet = (name) => "Hello, " + name;`

Arrow functions are used everywhere in Playwright (callbacks, event handlers).

For single expressions, you can omit the curly braces and return keyword.

Splitting Code Across Files:

As your project grows, you will have hundreds of lines of code.

Organize code by putting related functions in separate files.

Use `export` to make a function available to other files.

Use `import` to bring a function from another file into the current file.

This is how Page Objects and test helpers are structured in real projects.

Execution Flow Diagram



Syntax error in text

mermaid version 10.9.6

3. Implementation & Code Examples

helpers.js

TS

```
// Export functions so other files can use them
export function createTodoTitle() {
  const timestamp = Date.now();
  return "Todo-" + timestamp;
}

export function isValidTitle(title) {
  return title.length > 0 && title.length ≤ 100;
}

// --- File: main.js ---
// Import the functions from helpers.js
import { createTodoTitle, isValidTitle } from "./helpers.js";

const title = createTodoTitle();
console.log("Generated:", title);
console.log("Is valid:", isValidTitle(title));

// Arrow function example
const double = (n) => n * 2;
console.log("Double of 5:", double(5)); // 10
```

4. Best Practices (Do's and Don'ts)

Do

Give functions clear, descriptive names that explain what they do.

Use arrow functions for short, inline operations.

Keep each file focused on one responsibility.

Don't

Do not write functions that do too many things — split them up.

Do not forget the return keyword if your function needs to give back a value.

5. Chapter Summary

Functions are reusable blocks of code with inputs (parameters) and outputs (return values).

Arrow functions (=>) are a concise syntax used extensively in Playwright.

Use export/import to organize code across multiple files.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Write a function called multiply that takes two numbers and returns their product. Call it with 3 different pairs of numbers.

▼ 💡 Exercise 1 Solution

```
code CODE  
  
// Exercise 1 Solution:  
function multiply(a, b) {  
  return a * b;  
}  
  
console.log(multiply(3, 4));    // 12  
console.log(multiply(7, 8));    // 56  
console.log(multiply(10, 0));   // 0
```

Create two files: math-helpers.js (export an add and subtract function) and app.js (import and use them).

▼ 💡 Exercise 2 Solution

```
code CODE

// Exercise 2 Solution:
// --- File: math-helpers.js ---
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }

// --- File: app.js ---
import { add, subtract } from "./math-helpers.js";
console.log("5 + 3 =", add(5, 3));          // 8
console.log("10 - 4 =", subtract(10, 4)); // 6
```

🚀 Mini-Project

Build a string utilities module with functions: `capitalize(str)`, `truncate(str, maxLen)`, and `countWords(str)`. Export them and use them in a separate file.

▼ 💡 Mini-Project Solution

```
string-utils.js CODE

export function capitalize(str) {
  return str.charAt(0).toUpperCase() + str.slice(1);
}
export function truncate(str, maxLen) {
  if (str.length <= maxLen) return str;
  return str.slice(0, maxLen) + "...";
}
export function countWords(str) {
  return str.split(" ").filter(w => w.length > 0).length;
}

// --- File: use-utils.js ---
import { capitalize, truncate, countWords } from "./string-utils.js";
console.log(capitalize("hello"));           // Hello
console.log(truncate("A very long title", 10)); // A very lon...
console.log(countWords("Buy groceries today")); // 3
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Create `todo-helpers.js` — reusable functions for your automation project:

javascript

JS

```
export function generateTodoTitle() {
  return "Todo-" + Date.now();
}

export function formatTodoList(todos) {
  return todos.map((t, i) => (i + 1) + ". [" + (t.completed ? "x" : " ") + "] ")
};
}
```

This module will evolve into your Page Object helper layer.

8. Interview Q&A Preparation

Q1: What is the difference between a regular function and an arrow function?

Expected Answer: Arrow functions use concise syntax (=>) and inherit the this context from their parent scope. Regular functions bind their own this context. Arrow functions are preferred for callbacks and inline operations.

9. Chapter Cheat Sheet

code

CODE

```
function name(param) { return val; } // Regular\nconst fn = (p) => p * 2
```

✓ Completed Chapter 06 / 40

15%

PHASE 0 – PREREQUISITES

PHASE 0 – PREREQUISITES

CHAPTER 07

Reading Errors & Debugging

 15 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Identify common JavaScript error types
- ✓ Read and understand stack traces
- ✓ Use `console.log` to debug problems

CHAPTER 7 • Reading Errors & Debugging

1. Why This Matters

Errors are not failures — they are messages. Every professional developer reads errors hundreds of times per day. Learning to read them calmly is the single most important skill for self-sufficiency.

2. Conceptual Overview

When code breaks, the terminal prints an error message. This message is your best friend — it tells you exactly what went wrong and where.

Common Error Types:

SyntaxError: You made a typo. Missing bracket, extra comma, misspelled keyword.

ReferenceError: You used a variable name that does not exist.

TypeError: You tried to do something impossible (like calling `.click()` on `undefined`).

Reading a Stack Trace:

A stack trace is the list of lines printed after an error.

The first line tells you **WHAT** went wrong (the error type and message).

The second line tells you **WHERE** it happened (file name, line number, column number).

Always read from the top down.

Debugging with `console.log`:

When you do not understand why code behaves unexpectedly, add `console.log` statements to print values at different points.

This is called "print debugging" and every engineer uses it daily.

How to Google an Error:

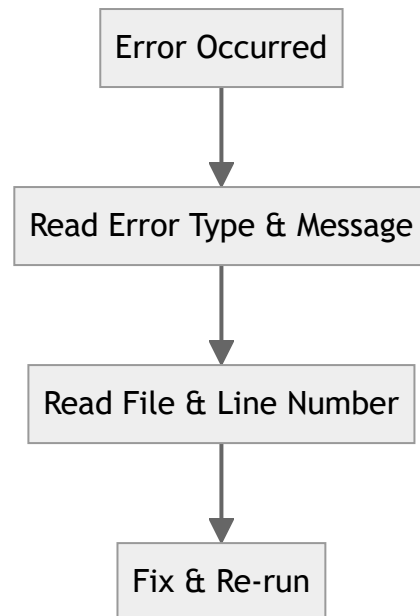
Copy the error **TYPE** and **MESSAGE** (not the file path).

Example: Search for "TypeError: Cannot read properties of undefined" not the full trace.

Add "JavaScript" or "Node.js" to your search.

Look for Stack Overflow results — they are usually the most helpful.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// ERROR 1: SyntaxError - Missing closing bracket
// if (score > 70 {           // ❌ Broken - missing )
//   console.log("Pass");
// }
// Fix:
if (score > 70) {             // ✅ Fixed - added )
  console.log("Pass");
}

// ERROR 2: ReferenceError - Variable not defined
// console.log(userName);    // ❌ ReferenceError: userName is not defined
// Fix: Declare the variable first
const userName = "Alice";
console.log(userName);       // ✅ Works

// ERROR 3: TypeError - Calling method on undefined
const user = { name: "Alice" };
// console.log(user.email.toUpperCase()); // ❌ TypeError: Cannot read property
// Fix: Check if property exists
if (user.email) {
  console.log(user.email.toUpperCase());
} else {
  console.log("No email set"); // ✅ Safe
}

// DEBUGGING: Print values to find bugs
function calculateTotal(items) {
  console.log("DEBUG - items received:", items); // Add this to inspect
  let total = 0;
  for (const item of items) {
    console.log("DEBUG - processing:", item);    // And this
    total += item.price;
  }
  return total;
}
```

4. Best Practices (Do's and Don'ts)

Do

Read error messages calmly — they are instructions, not punishments.

Always look at the line number in the stack trace first.

Use `console.log` to inspect variable values when debugging.

Don't

Do not delete your code and start over when you see an error — read the message first.

Do not ignore errors — they always have a cause and a fix.

5. Chapter Summary

`SyntaxError` = typo, `ReferenceError` = missing variable, `TypeError` = wrong operation.

Stack traces tell you the exact file and line where the error occurred.

`console.log` is your primary debugging tool — use it liberally.

6. Exercises & Mini-Project

Exercises

Intentionally write a program with a `SyntaxError`, a `ReferenceError`, and a `TypeError`. Run each one and practice reading the error messages.

▼ 💡 Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
// File 1 – syntax-error.js
// if (true {           // Run this – you'll see: SyntaxError: Unexpect
//   console.log("hi");
// }

// File 2 – reference-error.js
// console.log(myName); // ReferenceError: myName is not defined

// File 3 – type-error.js
// const data = undefined;
// data.toString();      // TypeError: Cannot read properties of undef

// Practice reading each error's line number and message!
```

Write a function that has a bug (wrong variable name). Add `console.log` statements to find and fix the bug.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
function greetUser(name) {
  // BUG: using 'username' instead of 'name'
  // console.log("Hello, " + username); // ReferenceError!

  // DEBUG: Add console.log to inspect
  console.log("DEBUG – parameter received:", name);

  // FIX: Use the correct variable name
  console.log("Hello, " + name);
}

greetUser("Alice");
```

🚀 Mini-Project

Build an error journal: create a file where you document 3 errors you encountered, what the error message said, and how you fixed them. This becomes your personal debugging reference.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution – error-journal.js
console.log("=== My Error Journal ===\n");

console.log("Error 1: SyntaxError");
console.log("Message: Unexpected token '{'");
console.log("Cause: Missing closing parenthesis in if statement");
console.log("Fix: Added ) before {\n");

console.log("Error 2: ReferenceError");
console.log("Message: myName is not defined");
console.log("Cause: Used a variable before declaring it");
console.log("Fix: Added const myName = 'Alice' before the console.log");

console.log("Error 3: TypeError");
console.log("Message: Cannot read properties of undefined");
console.log("Cause: Tried to call .toUpperCase() on an undefined object");
console.log("Fix: Added an if-check before accessing the property");
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Intentionally break your `plan.js` file and practice reading the error:

```
javascript JS

// Change this line to create a bug:
console.log(todoTitle); // ReferenceError – todoTitle is not defined

// Read the error message. Note the line number.
// Fix it by declaring the variable first:
const todoTitle = "Buy groceries";
console.log(todoTitle); // Now it works!
```

8. Interview Q&A Preparation

Q1: What is a stack trace and how do you read it?

Expected Answer: A stack trace is a list of function calls printed when an error occurs.

Read from top to bottom — the first line shows the error type and message, the second line shows the exact file, line number, and column where it happened.

9. Chapter Cheat Sheet

code

CODE

```
SyntaxError → Typo in code\nReferenceError → Missing variable\nTypeError
```

✓ Completed Chapter 07 / 40

18%

PHASE 0 – PREREQUISITES

PHASE 0 – PREREQUISITES

CHAPTER 08

Your First Playwright Test

 20 mins Reading Absolute Beginner Level **What You'll Master**

- ✓ Install Playwright in a project
- ✓ Write and run your first automated test
- ✓ Understand what happens when a test runs

CHAPTER 8 • Your First Playwright Test

1. Why This Matters

This is the moment everything clicks. You will write a test, run it, see a real browser open, watch it perform actions automatically, and see the result. This is the foundation for everything that follows.

2. Conceptual Overview

This chapter walks you through installing Playwright and running your very first automated test step by step.

Step 1: Install Playwright

Open your terminal in your project folder.

Run: `npm install -D @playwright/test`

Then run: `npx playwright install`

This downloads the test framework AND the browser engines (Chromium, Firefox, WebKit).

Step 2: Create Your Test File

Create a folder called `tests` inside your project.

Inside `tests`, create a file called `first.spec.ts` (the `.spec.ts` extension tells Playwright this is a test file).

Write a simple test that opens a website and checks the page title.

Step 3: Run Your Test

In your terminal, run: `npx playwright test`

Playwright will open a browser (in headless mode by default), visit the website, check the title, and report the result.

To SEE the browser open visually, run: `npx playwright test --headed`

Step 4: Understand What Happened

`npx playwright test` told the Playwright runner to find all `.spec.ts` files and execute them.

The runner launched a headless Chromium browser.

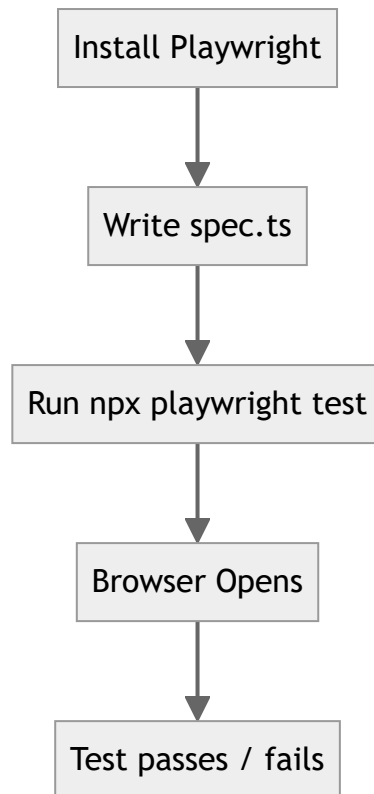
It navigated to the URL you specified.

It checked the page title against your expectation.

It reported PASS or FAIL in your terminal.

Congratulations — you just automated a browser!

Execution Flow Diagram



3. Implementation & Code Examples

tests/first.spec.ts

TS

```
// This is your very first Playwright test!

import { test, expect } from '@playwright/test';

test('verify TodoMVC page title', async ({ page }) => {
  // Step 1: Navigate to the TodoMVC demo app
  await page.goto('https://demo.playwright.dev/todomvc');

  // Step 2: Check that the page title contains "TodoMVC"
  await expect(page).toHaveTitle(/TodoMVC/);

  // Step 3: Verify the main heading is visible
  await expect(page.getByRole('heading', { name: 'todos' })).toBeVisible();
});

// To run this test, type in your terminal:
// npx playwright test
//
// To see the browser open:
// npx playwright test --headed
//
// Expected output:
// Running 1 test using 1 worker
//   ✓ first.spec.ts:5:5 › verify TodoMVC page title (1.2s)
//   1 passed (2.5s)
```

4. Best Practices (Do's and Don'ts)

Do

Run `npx playwright test --headed` the first time to see the browser in action.

Read the terminal output after every run — it tells you exactly what passed and failed.

Use `npx playwright test --ui` to open the interactive test viewer.

Don't

Do not skip `npx playwright install` — without it, there are no browser engines to run tests.

Do not worry about understanding every keyword (`async`, `await`, `expect`) right now — they are explained in later chapters.

5. Chapter Summary

Install Playwright with `npm install -D @playwright/test` and `npx playwright install`.

Create test files in a `tests/` folder with the `.spec.ts` extension.

Run tests with `npx playwright test`. Add `--headed` to see the browser.

6. Exercises & Mini-Project

Exercises

Write a second test in the same file that adds a todo item by typing in the input box and pressing Enter. Verify the todo appears in the list.

Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
test('add a new todo item', async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc');

  // Type a new todo in the input box and press Enter
  const input = page.getByPlaceholder('What needs to be done?');
  await input.fill('Buy groceries');
  await input.press('Enter');

  // Verify the todo appears in the list
  await expect(page.getByTestId('todo-title')).toHaveText('Buy grocer
});
```

Run your tests with `--headed` flag and watch the browser perform the actions.

▼ 💡 Exercise 2 Solution

code

CODE

```
# Exercise 2 Solution:  
# Simply run this command in your terminal:  
npx playwright test --headed  
# Watch the browser open, navigate to TodoMVC, and perform the action
```

🚀 Mini-Project

Write a small test suite (3 tests) that: (1) checks the page loads, (2) adds a todo and verifies it appears, (3) marks a todo as complete and verifies the strikethrough style.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution – tests/todomvc-basics.spec.ts
import { test, expect } from '@playwright/test';

test('page loads correctly', async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc');
  await expect(page).toHaveTitle(/TodoMVC/);
  await expect(page.getByRole('heading', { name: 'todos' })).toBeVisible();
});

test('add a todo item', async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc');
  const input = page.getByPlaceholder('What needs to be done?');
  await input.fill('Learn Playwright');
  await input.press('Enter');
  await expect(page.getByTestId('todo-title')).toHaveText('Learn Playwright');
});

test('complete a todo item', async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc');
  const input = page.getByPlaceholder('What needs to be done?');
  await input.fill('Complete this task');
  await input.press('Enter');

  // Click the toggle checkbox to mark as complete
  await page.getByRole('checkbox').click();

  // Verify the todo has the completed class
  await expect(page.getByTestId('todo-item')).toHaveClass(/completed/);
});
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: This IS the progressive project step! You just wrote your first real Playwright test against TodoMVC. From this point forward, every chapter will add a new capability to this test suite.

Your project folder should now look like:

```
code CODE
todomvc-automation/
├─ package.json
├─ node_modules/
├─ tests/
│   └─ first.spec.ts
└─ plan.js
```

8. Interview Q&A Preparation

Q1: What does `npx playwright test` do?

Expected Answer: It launches the Playwright test runner, which scans for `.spec.ts` files, starts browser engines, executes each test in an isolated browser context, and reports pass/fail results to the terminal.

9. Chapter Cheat Sheet

```
code CODE
npm install -D @playwright/test # Install\nnpx playwright install
```

✓ Completed Chapter 08 / 40

20%

PHASE 0 - PREREQUISITES

JAVASCRIPT FOR AUTOMATION

CHAPTER 09

Modern JS/TS Variables & Scoping

 15 mins Reading Beginner Level **What You'll Master**

- ✓ Understand lexical scoping in JS/TS
- ✓ Differentiate const, let, and var
- ✓ Avoid scope pollution in parallel tests

CHAPTER 9 • Modern JS/TS Variables & Scoping

1. Why This Matters

Using the incorrect variable scope in tests can cause shared memory leakage between parallel threads, leading to flaky test runs and unpredictable execution states.

2. Conceptual Overview

For manual testers transitioning to automated testing, variables are the starting point. In manual testing, you perform actions sequentially. In code, you store usernames, passwords, and URLs in variables.

Let's break down the main concepts step-by-step:

Variable Declaration:

const (Constant): Declares a read-only variable that cannot be reassigned. Use this for values that never change during execution, such as URLs, configuration options, and locator paths.

let: Declares a variable that can be updated or reassigned. Use this for loops, counters, or flags.

var: The legacy declaration type. Avoid this completely because it does not respect block boundaries and can leak data into other parts of your test, causing confusing errors when tests run in parallel.

Lexical and Block Scoping:

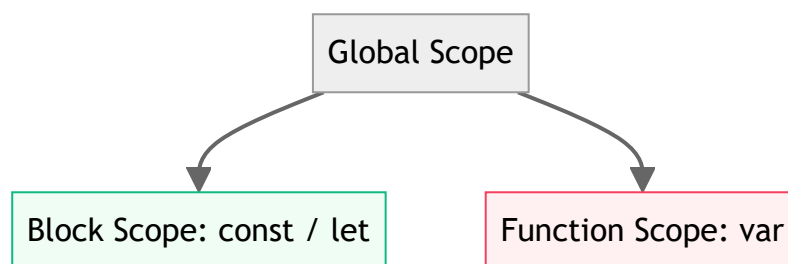
A block is code enclosed in curly braces {}.

When you declare a variable with `const` or `let`, it exists only inside those braces. Attempting to access it outside of them results in a `ReferenceError`.

Understanding `ReferenceErrors` and Stack Traces:

When a variable is accessed outside of its scope, the runtime throws a `ReferenceError`. A stack trace is a list of method calls that shows exactly where the error occurred (including file name, line number, and column number). Learning to read this trace tells you which scope was breached.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// ✅ Recommended Scoping
const BASE_URL = 'https://staging.enterprise.com'; // Immutable block-scope
let retryCounter = 0; // Block-scoped mutable variable

if (retryCounter === 0) {
  const tempId = 'SESSION-ID'; // Scoped strictly to this block
  console.log('Logged in with ID:', tempId);
  retryCounter++;
}

// console.log(tempId); // ❌ Throws ReferenceError!
// Stack trace analysis:
// ReferenceError: tempId is not defined
//   at Object.<anonymous> (tests/login.spec.ts:10:13)
// Analysis: Look at line 10, column 13. The compiler cannot find 'tempId' because it is not in the global scope.
```

4. Best Practices (Do's and Don'ts)

Do

Always default to const for variable definitions.

Declare loop counters using let to enforce block-scoping.

Read stack traces from the top down to locate the exact file and line of a scoping crash.

Don't

Never declare variables using var inside test suites.

Do not declare mutable variables globally in describe scopes.

5. Chapter Summary

Default to const for all variables to prevent accidental mutations.

Use let only when a variable needs to be reassigned (like loop counters).

Never use var inside modern test automation suites as it violates block scoping boundaries.

6. Exercises & Mini-Project

Exercises

Fix an assignment error inside a loop by replacing var with let.

▼ 💡 Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
// Before (broken):
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // Prints 3, 3, 3
}

// After (fixed):
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // Prints 0, 1, 2
}
```

Write a scoped helper function that dynamically returns credentials.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
function getCredentials(env: string) {
  if (env === 'staging') {
    return { user: 'staging_admin', pass: 'stg_pass' };
  }
  return { user: 'prod_admin', pass: 'prod_pass' };
}

const creds = getCredentials('staging');
console.log(creds.user); // staging_admin
```

Mini-Project

Build a credential state builder class that isolates environment scopes.

▼ Mini-Project Solution

```
code CODE

// Mini-Project Solution:
class CredentialBuilder {
  private env: string;
  constructor(env: string) { this.env = env; }

  getBaseUrl(): string {
    return this.env === 'prod' ? 'https://app.com' : 'https://staging'
  }

  getUser(): string {
    return this.env === 'prod' ? 'prod_user' : 'staging_user';
  }
}

const staging = new CredentialBuilder('staging');
console.log(staging.getBaseUrl()); // https://staging.app.com
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Declare scoped constants for the TodoMVC test configuration:

typescript

TS

```
const BASE_URL = "https://demo.playwright.dev/todomvc";
let todoCount = 0;

// Inside a test block – scoped correctly
if (todoCount === 0) {
  const firstTodo = "Learn scoping";
  console.log("Adding:", firstTodo);
  todoCount++;
}
// firstTodo is not accessible here – proper block scoping!
```

8. Interview Q&A Preparation

Q1: What is the main danger of using var inside async test loops?

Expected Answer: Because var is function-scoped, the loop variable shares the same memory reference. By the time async actions execute, the loop has completed, and all iterations will point to the final value, causing race conditions.

9. Chapter Cheat Sheet

code

CODE

```
const key = val; // Immutable
let varName = val; // Mutable
```

✓ Completed Chapter 09 / 40

23%

JAVASCRIPT FOR AUTOMATION

JAVASCRIPT FOR AUTOMATION

CHAPTER 10

Functions, Closures, and Callbacks

 15 mins Reading Beginner Level **What You'll Master**

- ✓ Master arrow function expressions
- ✓ Understand callbacks in event handling
- ✓ Write lexical closures

CHAPTER 10 • Functions, Closures, and Callbacks

1. Why This Matters

Playwright utilizes callback arguments extensively (e.g. inside `page.route` checks or `expect.poll`). Knowing how lexical scopes bind values resolves evaluation bugs.

2. Conceptual Overview

To transition from writing basic linear scripts to creating reusable helpers, you must understand how functions carry information. In automation, a function is a reusable block of instructions.

Let's break these down step-by-step:

Arrow Functions `() => {}`:

These are a modern, concise way to write functions in JavaScript.

Instead of writing `function() { return page.click(); }`, you write `() => page.click()`.

They are extremely useful in class methods because they do not bind their own `this` context, instead inheriting it lexically.

Callbacks:

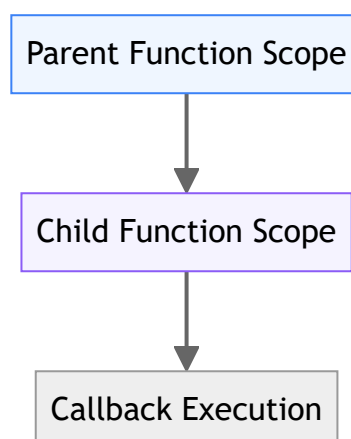
A callback is a function passed as an argument to another function, to be executed later.

In Playwright, when we wait for a condition or inspect network routes, we pass a callback specifying what to check once the event occurs.

Lexical Closures:

A closure is created when an inner function retains access to its parent's outer variables, even after the parent function has finished executing. Think of it as a backpack: the inner function carries the parent's variables wherever it goes.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Callback: Passed as an inline arrow function to page.route
await page.route('**/api/config', (route) => {
  route.fulfill({ status: 200, body: JSON.stringify({ theme: 'dark' }) });
});

// Lexical Closure: Returns a function retaining access to 'minScore'
function createScoreChecker(minScore: number) {
  return (score: number) => score >= minScore;
}
const passesLimit = createScoreChecker(80);
console.log(passesLimit(85)); // true
```

4. Best Practices (Do's and Don'ts)



Do

Use arrow functions for lightweight, inline callback parameters.

Capture snapshot variables in closures for dynamic waits.



Don't

Do not lose contextual bindings by misusing traditional function closures.

5. Chapter Summary

Master arrow functions to write concise, inline test callbacks.

Understand how callbacks allow Playwright functions to execute steps asynchronously.

Leverage closures to retain access to outer variables within async event scopes.

6. Exercises & Mini-Project



Exercises

Write a function returning a closure that filters page logs by key strings.

▼ 💡 Exercise 1 Solution

```
code CODE  
  
// Exercise 1 Solution:  
function createLogFilter(keyword: string) {  
  return (logEntry: string) => logEntry.toLowerCase().includes(keyword)  
}  
  
const errorFilter = createLogFilter('error');  
const logs = ['Info: started', 'Error: timeout', 'Error: crash', 'Info: ended'];  
const errors = logs.filter(errorFilter);  
console.log(errors); // ['Error: timeout', 'Error: crash']
```

Refactor a standard function block callback into a concise arrow expression.

▼ 💡 Exercise 2 Solution

```
code CODE  
  
// Exercise 2 Solution:  
// Before (standard function):  
page.route('**/api', function(route) {  
  return route.fulfill({ status: 200, body: '{}' });  
});  
  
// After (arrow function):  
page.route('**/api', (route) => route.fulfill({ status: 200, body: '{}' }));
```

🚀 Mini-Project

Create a dynamic logger middleware wrapper using closures.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
function createLogger(prefix: string) {
  let count = 0;
  return (message: string) => {
    count++;
    console.log([' ' + prefix + ' #' + count + ' ] ' + message);
  };
}

const testLog = createLogger('TEST');
testLog('Started login flow'); // [TEST #1] Started login flow
testLog('Clicked submit');     // [TEST #2] Clicked submit
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Write a closure that generates unique todo titles with an incrementing counter:

```
typescript TS

function createTodoGenerator(prefix: string) {
  let counter = 0;
  return () => {
    counter++;
    return prefix + "-" + counter + "-" + Date.now();
  };
}

const genTodo = createTodoGenerator("Task");
console.log(genTodo()); // Task-1-1719999 ...
console.log(genTodo()); // Task-2-1719999 ...
```


8. Interview Q&A Preparation

Q1: How do arrow functions handle the this context inside Page Objects?

Expected Answer: Arrow functions do not bind their own this context; they inherit it lexically from the enclosing scope. This is useful in classes to preserve access to class properties.

9. Chapter Cheat Sheet

code

CODE

```
const myFunc = () => { ... }; // Arrow function
```

✓ Completed Chapter 10 / 40

25%

JAVASCRIPT FOR AUTOMATION

JAVASCRIPT FOR AUTOMATION

CHAPTER 11

Asynchronous JS: Promises & Event Loop

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Understand JavaScript single-threaded event loop
- ✓ Master Promises states
- ✓ Implement async/await configurations

CHAPTER 11 • Asynchronous JS: Promises & Event Loop

1. Why This Matters

Web automation actions are asynchronous network calls. Forgetting to await a Playwright execution returns a pending Promise, causing the test to pass or fail prematurely.

2. Conceptual Overview

JavaScript is a single-threaded execution runtime. It handles concurrent operations by delegating tasks via Promises. In Playwright, it is vital to understand the execution boundary:

Execution Boundary:

Node.js Process: Runs your test script, manages configuration, runs assertions, and schedules queues.

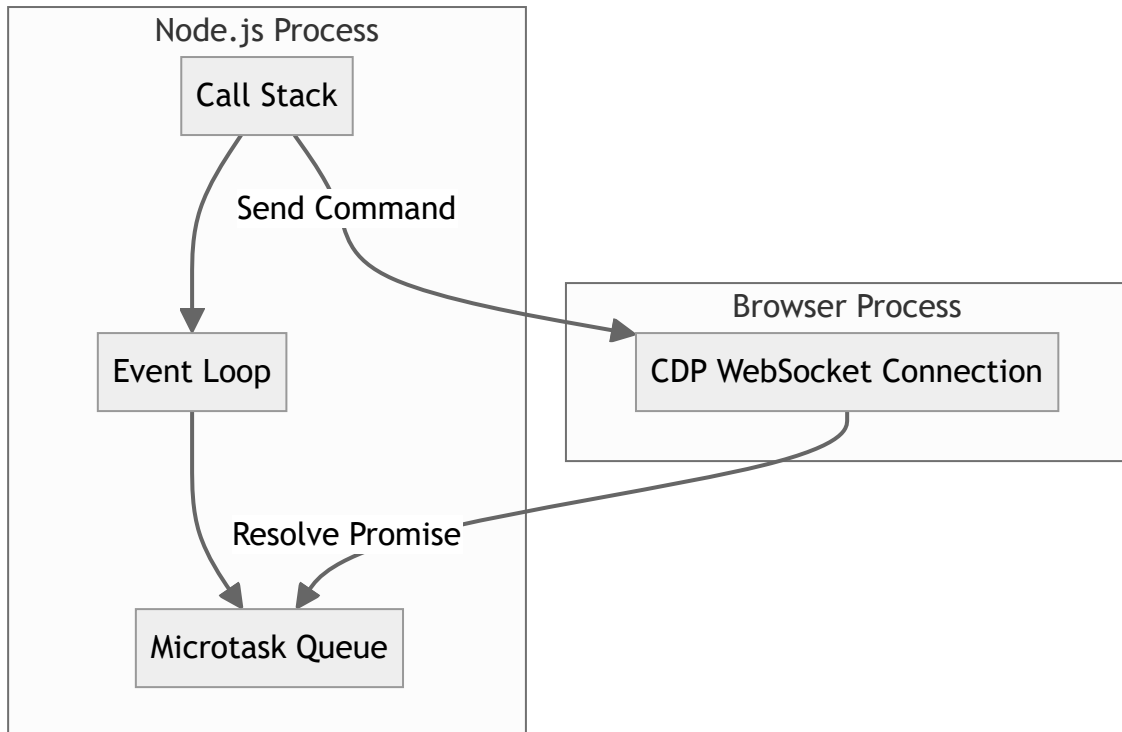
Browser Process (Chromium/WebKit): Renders the web application page under test.

Communication Bridge: Node.js and the browser communicate asynchronously over a WebSocket connection using the Chrome DevTools Protocol (CDP).

The Async/Await Queue:

When you invoke `await page.click()`, Node.js sends the click command over the WebSocket connection to the browser and registers a pending Promise. The Event Loop suspends the next line in Node.js until the browser signals that the click completed, which resolves the Promise. If you omit `await`, Node.js fires the event over the socket and immediately continues to the next line before the browser can execute the operation, causing runtime sync failures.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

// ✅ Recommended Asynchronous Execution
test('verify details page', async ({ page }) => {
  await page.goto('/dashboard');
  const detailsBtn = page.getByRole('button', { name: 'Details' });
  await detailsBtn.click(); // Pauses Node.js execution thread until browser c
});
  
```

4. Best Practices (Do's and Don'ts)

Do

Precede every browser action with the `await` keyword.

Return Promises from helper functions that interact with the page.

Don't

Never omit `await` on assertions or page actions.

Avoid using `.then()` and `async/await` syntax in the same code block.

5. Chapter Summary

Understand that JS is single-threaded and delegates async tasks via the Event Loop.

Always prepend asynchronous page interactions with the `await` keyword.

Differentiate pending, fulfilled, and rejected promise states in assertions.

6. Exercises & Mini-Project

Exercises

Fix a test that exits instantly because `page.goto` is not awaited.

Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
test('fix unawaited goto', async ({ page }) => {
  // Before (broken): page.goto('/dashboard'); – exits instantly
  // After (fixed):
  await page.goto('/dashboard');
  await expect(page).toHaveURL(/dashboard/);
});
```

Write a custom async function that waits for an API call and verify the result.

▼ 💡 Exercise 2 Solution

code

CODE

```
// Exercise 2 Solution:
async function waitForApiResult(page: Page, endpoint: string) {
  const response = await page.request.get(endpoint);
  const data = await response.json();
  return data;
}

test('verify API data', async ({ page }) => {
  const result = await waitForApiResult(page, '/api/status');
  expect(result.status).toBe('healthy');
});
```

🚀 Mini-Project

Build a custom promise-based wait utility that checks system state.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
async function waitForCondition(
  checkFn: () => Promise<boolean>,
  timeoutMs = 5000,
  intervalMs = 500
): Promise<void> {
  const start = Date.now();
  while (Date.now() - start < timeoutMs) {
    if (await checkFn()) return;
    await new Promise(r => setTimeout(r, intervalMs));
  }
  throw new Error('Condition not met within ' + timeoutMs + 'ms');
}
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Understand why `await` is required for every Playwright action:

typescript

TS

```
test("add todo with proper awaits", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Learn async/await"); // Must await!
  await input.press("Enter");           // Must await!
  // Without await, the test would exit before the browser acts
});
```

🎯 8. Interview Q&A Preparation

Q1: Why does forgetting `await` on a `page` action cause flaky tests?

Expected Answer: Without `await`, the test runner proceeds to the next line immediately. The target element might not be located, clicked, or verified in time before the browser context is torn down.

9. Chapter Cheat Sheet

code

CODE

```
async function run() { await page.click(); }
```

✓ Completed Chapter 11 / 40

28%

JAVASCRIPT FOR AUTOMATION

TYPESCRIPT FOR AUTOMATION

CHAPTER 12

TypeScript Types, Interfaces, & Generics

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Define strict TypeScript interfaces
- ✓ Implement type unions
- ✓ Utilize Generics for API utilities

CHAPTER 12 • TypeScript Types, Interfaces, & Generics

1. Why This Matters

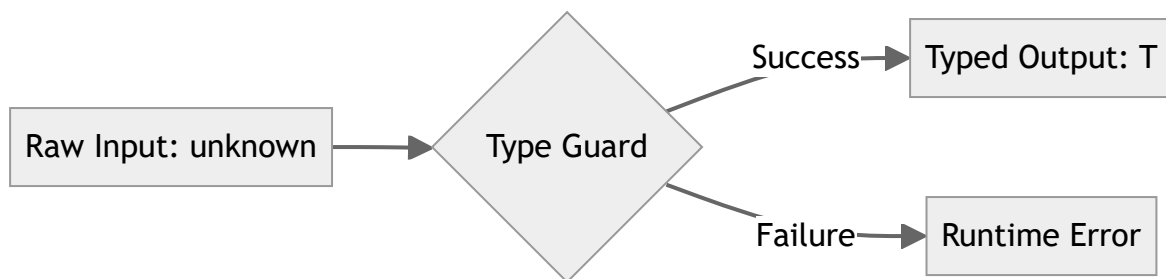
Using any types bypasses compiler validations. Type-safe test scripts catch structural bugs, such as missing properties in request payloads, during local compilations.

2. Conceptual Overview

TypeScript extends JavaScript by adding compiler-level types. Using interfaces, union types, and generic interfaces allows developers to structure page methods, response objects, and payload data factories safely.

However, a common anti-pattern in TypeScript is type casting using the `as T` operator (e.g., `res.json() as User`). This forces the compiler to ignore validation, which overrides safety. If the API schema changes at runtime, the compiler will not catch it, causing silent errors and contract drift. To prevent this, generics should be combined with runtime validation functions that inspect and verify the object structure before return.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

interface APIResponse<T> {
  data: T;
  statusCode: number;
}

interface UserProfile {
  username: string;
  email: string;
}

// Safe validation function to ensure the runtime object has required properties
function isUserProfile(data: any): data is UserProfile {
  return (
    data &&
    typeof data.username === 'string' &&
    typeof data.email === 'string'
  );
}

// Generic API response parser with strict runtime verification
function parseResponseSafely<T>(
  resBody: unknown,
  validator: (data: unknown) => data is T,
  status: number
): APIResponse<T> {
  if (validator(resBody)) {
    return {
      data: resBody, // Compiler narrows type safely from unknown to T
      statusCode: status
    };
  }
  throw new Error('API contract drift detected! Runtime payload does not match');
}

```

4. Best Practices (Do's and Don'ts)

 Do

Define interfaces for all API response payloads.

Combine Generic functions with runtime type-guards to validate payload shapes safely.

 Don't

Avoid using the any type fallback in production code.

Never use raw "as T" casting on dynamic network responses without a validation safeguard.

 5. Chapter Summary

Declare strict TypeScript interfaces to validate test payload configurations.

Eliminate any from the codebase to enable full compiler-level validation.

Use Generics to construct reusable, type-safe API helper functions.

 6. Exercises & Mini-Project Exercises

Create a type-safe checkout credentials interface and use it in a mock login page.

 Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
interface CheckoutCredentials {
  cardNumber: string;
  expiry: string;
  cvv: string;
  billingName: string;
}

const testCard: CheckoutCredentials = {
  cardNumber: '4111222233334444',
  expiry: '12/28',
  cvv: '123',
  billingName: 'Test User'
};
```

Write a generic function that parses database row columns safely.

▼ 💡 **Exercise 2 Solution**

code

CODE

```
// Exercise 2 Solution:
function parseDbRow<T>(row: unknown, validator: (data: unknown) => da
  if (validator(row)) return row;
  throw new Error('Row does not match expected schema');
}

interface ProductRow { id: number; name: string; price: number; }
function isProductRow(data: unknown): data is ProductRow {
  const d = data as any;
  return typeof d?.id === 'number' && typeof d?.name === 'string' &&
}
```

🚀 **Mini-Project**

Build a generic API validation middleware using TypeScript models.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
import { z } from 'zod';

const ApiResponseSchema = z.object({
  data: z.object({ id: z.number(), name: z.string() }),
  statusCode: z.number()
});

type ApiResponse = z.infer<typeof ApiResponseSchema>;

function validateResponse(raw: unknown): ApiResponse {
  return ApiResponseSchema.parse(raw);
}
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Define a TypeScript interface for TodoMVC data models:

typescript

TS

```
interface TodoItem {
  title: string;
  completed: boolean;
}

interface TodoAppState {
  todos: TodoItem[];
  filter: "all" | "active" | "completed";
}

const expectedState: TodoAppState = {
  todos: [{ title: "Learn TypeScript", completed: false }],
  filter: "all"
};
```

8. Interview Q&A Preparation

Q1: Why are Generics preferred over any in utility methods?

Expected Answer: Generics preserve the original type throughout execution. The calling code gets full autocompletion and compiler validation based on the specific type passed, which any disables.

9. Chapter Cheat Sheet

code

CODE

```
interface User { id: number; }
```

✓ Completed Chapter 12 / 40

30%

TYPESCRIPT FOR AUTOMATION

PLAYWRIGHT BASICS

CHAPTER 13

Installation & Core Architecture

 15 mins Reading Beginner Level **What You'll Master**

- ✓ Understand Browser, BrowserContext, and Page hierarchy
- ✓ Explain Chrome DevTools Protocol (CDP) interactions
- ✓ Configure basic test runner settings

CHAPTER 13 • Installation & Core Architecture

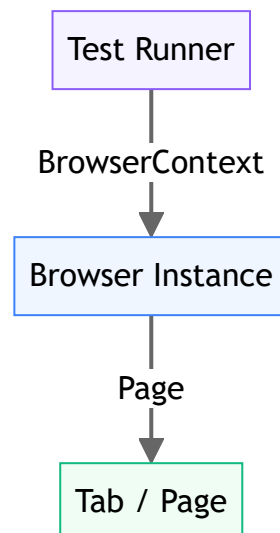
1. Why This Matters

Understanding browser context isolation explains why tests run concurrently without colliding, maximizing speed and efficiency.

2. Conceptual Overview

Playwright communicates with browser engines directly using the Chrome DevTools Protocol (CDP) over WebSockets. This removes the Selenium WebDriver REST API translation layer, enabling fast, bidirectional commands and native network interception. Because WebSocket protocols govern communications, Playwright provides APIs to intercept, inspect, and log all WebSocket traffic sent between the client application and browser engine.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Listen to page WebSocket connections and log client frames
test('track WebSocket messages', async ({ page }) => {
  page.on('websocket', (ws) => {
    console.log('WebSocket connection opened: ' + ws.url());

    // Extract and log frames received from the server
    ws.on('framereceived', (frame) => {
      const payloadStr = typeof frame.payload === 'string'
        ? frame.payload
        : frame.payload.toString('utf-8');
      console.log('Frame Received: ' + payloadStr);
    });

    // Extract and log frames sent from the client
    ws.on('framesent', (frame) => {
      const payloadStr = typeof frame.payload === 'string'
        ? frame.payload
        : frame.payload.toString('utf-8');
      console.log('Frame Sent: ' + payloadStr);
    });
  });

  await page.goto('/realtime-chat');
});
```

4. Best Practices (Do's and Don'ts)

Do

Leverage BrowserContexts to isolate sessions.

Use the page.on("websocket") event emitter to log active web socket packet payloads.

Don't

Do not share state between contexts.

Avoid running multiple browsers in a single thread manually.

5. Chapter Summary

Understand Browser, BrowserContext, and Page hierarchy.

Leverage BrowserContexts to isolate cookies, cache, and state per test.

Explain how CDP WebSocket communication replaces REST WebDriver.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Install Playwright inside a clean directory and run the initialization wizard.

▼ 💡 Exercise 1 Solution

```
code CODE
# Exercise 1 Solution:
mkdir my-playwright-project
cd my-playwright-project
npm init -y
npm install -D @playwright/test
npx playwright install
npx playwright test # Run the example tests
```

Add a new browser configuration project to playwright.config.ts.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution – playwright.config.ts:
import { defineConfig, devices } from '@playwright/test';
export default defineConfig({
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } }
  ]
});
```

Mini-Project

Write a runner script that logs WebSocket communication protocols.

▼ Mini-Project Solution

```
code CODE

// Mini-Project Solution:
page.on('websocket', (ws) => {
  console.log('WebSocket URL:', ws.url());
  ws.on('framereceived', (frame) => {
    const payload = typeof frame.payload === 'string' ? frame.payload
    console.log('Received:', payload);
  });
  ws.on('framesent', (frame) => {
    const payload = typeof frame.payload === 'string' ? frame.payload
    console.log('Sent:', payload);
  });
});
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Explore the Browser → Context → Page hierarchy by creating multiple contexts:

typescript

TS

```
test("understand context isolation", async ({ browser }) => {
  const context1 = await browser.newContext();
  const context2 = await browser.newContext();
  const page1 = await context1.newPage();
  const page2 = await context2.newPage();
  // page1 and page2 have completely separate cookies and storage
  await page1.goto("https://demo.playwright.dev/todomvc");
  await page2.goto("https://demo.playwright.dev/todomvc");
  await context1.close();
  await context2.close();
});
```

8. Interview Q&A Preparation

Q1: What is the structural difference between a Browser and a BrowserContext?

Expected Answer: A Browser represents the physical browser instance. A BrowserContext is an isolated session within that browser, similar to an Incognito window. It holds independent cookies, local storage, and caches.

9. Chapter Cheat Sheet

code

CODE

```
npx playwright install // Install browsers
```

✓ Completed Chapter 13 / 40

PLAYWRIGHT BASICS

33%

PLAYWRIGHT BASICS

CHAPTER 14

Navigating & Basic Actions

 15 mins Reading Beginner Level **What You'll Master**

- ✓ Master page navigation events
- ✓ Differentiate action methods
- ✓ Understand page actionability sequence

CHAPTER 14 • Navigating & Basic Actions

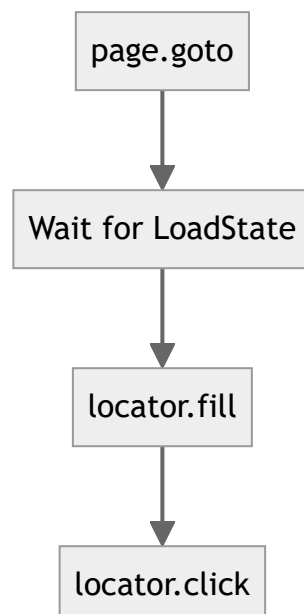
1. Why This Matters

Page interactions require elements to meet actionability standards. If elements are hidden or blocked, actions fail, signaling design bugs.

2. Conceptual Overview

Playwright does not click blindly. For every action, it checks if the target element is attached, visible, stable, enabled, and not covered. However, a major source of flakiness in Single Page Applications (SPAs) and micro-frontends is the "Hydration click gap". In modern frontend frameworks, elements are drawn on screen quickly (Server-Side Rendered or Static HTML) but their JavaScript event handlers have not finished downloading or executing. When this occurs, the element passes all actionability checks, but the click event is lost because the click listener is not bound yet. To solve this, tests should wait for specific hydration attributes (e.g., checking data attributes like `data-hydrated="true"`) rather than executing repetitive clicks that could trigger duplicate network requests if the button is partially bound.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
test('execute user interactions with hydration safety', async ({ page }) => {
  await page.goto('/form');

  const submitBtn = page.getByRole('button', { name: 'Submit' });

  // Wait for explicit hydration attribute before execution
  await expect(submitBtn).toHaveAttribute('data-hydrated', 'true', { timeout: 1000 });
  await submitBtn.click();

  await expect(page.getByText('Form Submitted')).toBeVisible();
});
```

4. Best Practices (Do's and Don'ts)

Do

Use the correct action method (.check(), .fill(), .hover()) instead of generic clicks.

Mitigate hydration click gaps in SPAs by waiting for hydration states.

Don't

Do not insert arbitrary page timeouts after navigations.

Avoid forcing clicks unless absolutely necessary.

Do not wrap click operations in repeating toPass retry loops as they can cause duplicate submissions.

5. Chapter Summary

Master Playwright's actionability checks (visible, stable, enabled).

Use page.goto and direct action methods instead of blind clicks.

Avoid hardcoded timers; leverage auto-waiting on load states.

6. Exercises & Mini-Project

Exercises

Write a test that navigates to a form, fills inputs, and clicks submit.

▼ 💡 Exercise 1 Solution

```
code CODE  
  
// Exercise 1 Solution:  
test('fill and submit a form', async ({ page }) => {  
  await page.goto('/form');  
  await page.getByLabel('Username').fill('testuser');  
  await page.getByLabel('Password').fill('secret123');  
  await page.getByRole('button', { name: 'Submit' }).click();  
  await expect(page.getByText('Welcome')).toBeVisible();  
});
```

Intercept navigation load states using `page.waitForLoadState`.

▼ 💡 Exercise 2 Solution

```
code CODE  
  
// Exercise 2 Solution:  
test('intercept navigation states', async ({ page }) => {  
  await page.goto('/dashboard');  
  await page.waitForLoadState('networkidle');  
  console.log('Network idle – all resources loaded');  
  await page.waitForLoadState('domcontentloaded');  
  console.log('DOM ready');  
});
```

🚀 Mini-Project

Build an interactive form checking page and validate input states.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
test('interactive form with hydration check', async ({ page }) => {
  await page.goto('/checkout');
  const submitBtn = page.getByRole('button', { name: 'Pay Now' });
  await expect(submitBtn).toHaveAttribute('data-hydrated', 'true', {
  await page.getByLabel('Name').fill('Alice');
  await page.getByLabel('Card').fill('4111222233334444');
  await submitBtn.click();
  await expect(page.getByText('Payment confirmed')).toBeVisible();
});
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Navigate to TodoMVC and add a todo using page actions:

```
typescript TS

test("add todo using fill and keyboard", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Navigate and act");
  await input.press("Enter");
  await expect(page.getByTestId("todo-title")).toHaveText("Navigate and act");
});
```

🎯 8. Interview Q&A Preparation

Q1: What does Playwright verify before clicking a button?

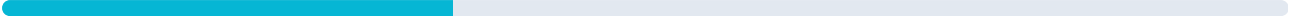
Expected Answer: It checks if the button is attached to the DOM, visible on screen, stable (not animating), enabled (not disabled), and can receive pointer events at the click coordinates.

📄 9. Chapter Cheat Sheet

code

CODE

```
await page.goto("/url");  
await locator.click();
```



✓ Completed Chapter 14 / 40

35%

PLAYWRIGHT BASICS

PLAYWRIGHT BASICS

CHAPTER 15

Locators & Selector Engines

 20 mins Reading Beginner Level **What You'll Master**

- ✓ Apply the Locator Priority Ladder rules
- ✓ Chain locators effectively
- ✓ Resolve strict mode violations cleanly

CHAPTER 15 • Locators & Selector Engines

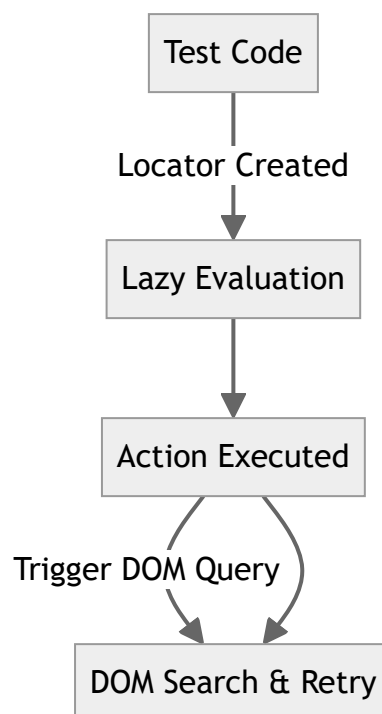
1. Why This Matters

Writing fragile CSS or XPath selectors leads to flaky tests. Accessibility-first locators make your test suite highly resilient to layout changes.

2. Conceptual Overview

Locators are queries that search the page. They are lazily evaluated: they do not query the DOM when created, but run when an action is called. They automatically retry if the DOM updates, eliminating stale element exceptions. In design, you should construct chained queries starting from an accessible container role to avoid brittle layout selections.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Locators chaining following the priority accessibility ladder
const checkoutForm = page.getByRole('form', { name: 'Checkout' });
const creditCardInput = checkoutForm.getByRole('textbox', { name: 'Credit Card' });
await creditCardInput.fill('4111222233334444');
```

🏆 4. Best Practices (Do's and Don'ts)

🟢 Do

Query using accessible roles (`getByRole`) first.

Filter list items using `.filter()` based on text content.

🔴 Don't

Avoid using raw index numbers like `.nth(0)` for dynamic lists.

Do not write long CSS paths or brittle XPath's.

📄 5. Chapter Summary

Follow the Locator Priority Ladder (`getByRole`, `getByLabel`, etc.).

Rely on lazy evaluation: queries run only when actions execute.

Solve strict mode violations by narrow chaining or filtering.

🏋️ 6. Exercises & Mini-Project

🎯 Exercises

Refactor a selector path from an XPath string to a semantic `getByRole` locator.

🔽 💡 Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
// Before (XPath):
// page.locator('//div[@class="main"]/ul/li[1]/button');

// After (semantic getByRole):
const editBtn = page.getByRole('listitem').first().getByRole('button')
```

Create a nested search loop that clicks edit buttons inside dynamic table rows.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
test('click edit on each table row', async ({ page }) => {
  const rows = page.getByRole('row');
  const count = await rows.count();
  for (let i = 1; i < count; i++) {
    const row = rows.nth(i);
    await row.getByRole('button', { name: 'Edit' }).click();
    await page.getByRole('button', { name: 'Cancel' }).click();
  }
});
```

🚀 Mini-Project

Write a script that sweeps a product listing page and compiles prices.

▼ 💡 Mini-Project Solution

```
code CODE
// Mini-Project Solution:
test('sweep product prices', async ({ page }) => {
  await page.goto('/products');
  const prices = await page.getByRole('listitem').allTextContents();
  const numericPrices = prices.map(p => parseFloat(p.replace('$', '')));
  console.log('All prices:', numericPrices);
  console.log('Total:', numericPrices.reduce((a, b) => a + b, 0));
});
```

📺 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Locate todos using semantic locators and chaining:

typescript

TS

```

test("locate todos with getByRole", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Task A");
  await input.press("Enter");
  await input.fill("Task B");
  await input.press("Enter");

  // Locate by test ID
  const todos = page.getByTestId("todo-title");
  await expect(todos).toHaveCount(2);

  // Filter to find specific todo
  const taskB = page.getByRole("listitem").filter({ hasText: "Task B" });
  await expect(taskB).toBeVisible();
});

```

8. Interview Q&A Preparation

Q1: Why are locators called "lazy" in Playwright?

Expected Answer: Locators do not query the browser when declared. They only search the DOM when an action (like click or fill) is executed, ensuring the latest page state is queried.

9. Chapter Cheat Sheet

code

CODE

```
const btn = page.getByRole("button", { name: "Save" });
```

✓ Completed Chapter 15 / 40

38%

PLAYWRIGHT BASICS

PLAYWRIGHT BASICS

CHAPTER 16

Web-First Assertions

 20 mins Reading Beginner Level **What You'll Master**

- ✓ Master Web-First assertions polling rules
- ✓ Implement soft assertions
- ✓ Configure polling loops using `expect.poll`

CHAPTER 16 • Web-First Assertions

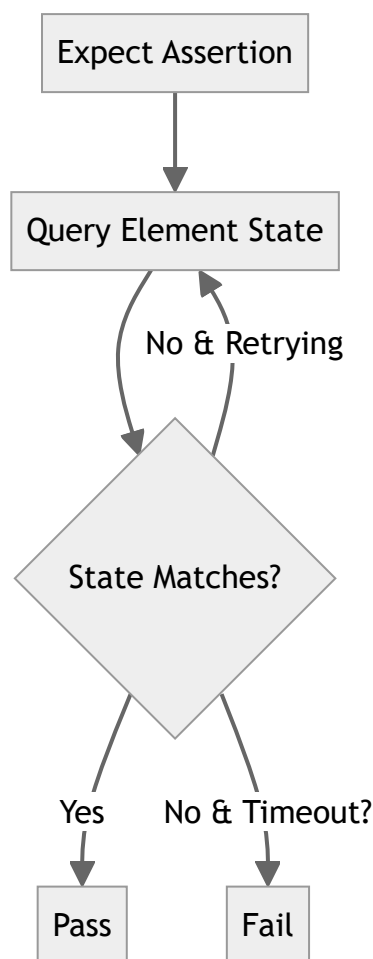
1. Why This Matters

Using assertions that do not retry leads to race conditions. Web-First assertions wait for elements to match, ensuring robust tests.

2. Conceptual Overview

Playwright introduces Web-First assertions. By passing locators directly to expect, assertions automatically check conditions in a loop until the state matches or the timeout limit is reached. For checking background server tasks, databases, or REST APIs that update outside the DOM, `expect.poll` allows running real asynchronous checks continuously.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Asynchronous database state verification using API polling
await expect.poll(async () => {
  const response = await page.request.get('/api/job-status/456', {
    headers: {
      'Authorization': 'Bearer token123',
      'Accept': 'application/json'
    }
  });
  if (response.status() !== 200) {
    throw new Error('API server returned error code: ' + response.status());
  }
  const result = await response.json();
  return result.status;
}, {
  timeout: 10000,
  intervals: [1000, 2000]
}).toBe('SUCCESS');
```

4. Best Practices (Do's and Don'ts)

Do

Always pass locator objects directly to expect().

Use expect.soft() when validating secondary elements.

Don't

Avoid assertions against static variables, e.g. expect(await loc.isVisible()).toBe(true).

5. Chapter Summary

Pass locator objects directly to expect for auto-retry validations.

Use expect.soft to track secondary failures without halting runs.

Configure expect.poll to wait dynamically for database/API states.

6. Exercises & Mini-Project

Exercises

Write a validation suite using expect.soft to inspect card layout details.

▼ 💡 Exercise 1 Solution

```
code CODE  
  
// Exercise 1 Solution:  
test('soft assert card layout', async ({ page }) => {  
  await page.goto('/dashboard');  
  const card = page.locator('.product-card').first();  
  await expect.soft(card.getByRole('img')).toBeVisible();  
  await expect.soft(card.getByRole('heading')).toHaveText(/.+/);  
  await expect.soft(card.getByText('$')).toBeVisible();  
});
```

Configure `expect.toPass` to repeat a refresh action until a result appears.

▼ 💡 Exercise 2 Solution

```
code CODE  
  
// Exercise 2 Solution:  
test('wait for data refresh', async ({ page }) => {  
  await page.goto('/dashboard');  
  await expect(async () => {  
    await page.getByRole('button', { name: 'Refresh' }).click();  
    await expect(page.getByTestId('result')).toHaveText(/data-loaded/  
  }).toPass({ timeout: 15000, intervals: [1000, 2000] });  
});
```

🚀 Mini-Project

Build a dashboard checker page with soft assertions.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
test('dashboard soft assertion checker', async ({ page }) => {
  await page.goto('/dashboard');
  const cards = page.locator('.dashboard-card');
  const count = await cards.count();
  for (let i = 0; i < count; i++) {
    await expect.soft(cards.nth(i).getByRole('heading')).toBeVisible();
    await expect.soft(cards.nth(i).getByText('$')).toBeVisible();
  }
});
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Assert todo count and visibility using Web-First assertions:

```
typescript TS

test("verify todo state with assertions", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Learn assertions");
  await input.press("Enter");

  // Web-First: auto-retries until condition matches
  await expect(page.getByTestId("todo-title")).toHaveText("Learn assertions");
  await expect(page.getByTestId("todo-title")).toHaveCount(1);
  await expect(page.getByText("1 item left")).toBeVisible();
});
```

🎯 8. Interview Q&A Preparation

Q1: What is the main benefit of Web-First assertions over basic assertions?

Expected Answer: Web-First assertions auto-retry. They query the element state continuously up to a timeout, resolving race conditions when elements render asynchronously.

9. Chapter Cheat Sheet

code

CODE

```
await expect(locator).toBeVisible();
```

✓ Completed Chapter 16 / 40

40%

PLAYWRIGHT BASICS

PLAYWRIGHT BASICS

CHAPTER 17

Test Organization & Hooks

 15 mins Reading Beginner Level **What You'll Master**

- ✓ Create describe scopes
- ✓ Implement lifecycle hooks
- ✓ Configure serial and parallel run settings

CHAPTER 17 • Test Organization & Hooks

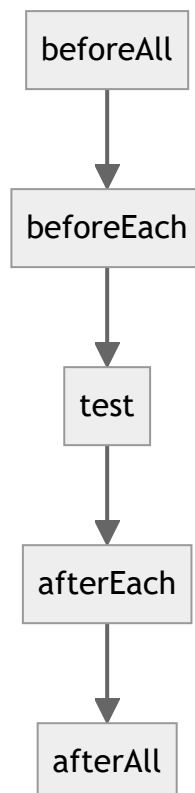
1. Why This Matters

Correct test grouping isolates specs, shares preconditions cleanly, and ensures tests run safely in parallel.

2. Conceptual Overview

Use `test.describe` blocks to group related tests. Lifecycle hooks (`beforeAll`, `beforeEach`, `afterEach`, `afterAll`) set up preconditions and clean up environments, keeping tests decoupled.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
test.describe('Payment Funnel', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/checkout');
  });

  test('validate credit card input', async ({ page }) => {
    const cardInput = page.getByRole('textbox', { name: 'Card Details' });
    await cardInput.fill('4111222233334444');
    console.log('Action complete: card number filled.');
  });
});
```

4. Best Practices (Do's and Don'ts)

Do

Group tests by feature areas.

Declare beforeEach hooks to navigate to base URLs.

Don't

Do not share runtime data states across tests.

Avoid running tests serially unless strictly necessary.

5. Chapter Summary

Group related tests inside describe blocks.

Use hook lifecycles (beforeEach, afterAll) to isolate setups.

Enforce serial or parallel structures depending on data dependence.

6. Exercises & Mini-Project

Exercises

Create a nested describe block mapping cart addition and payment actions.

▼ 💡 Exercise 1 Solution

```
code CODE

// Exercise 1 Solution:
test.describe('Cart', () => {
  test.describe('Add to Cart', () => {
    test('add single item', async ({ page }) => {
      await page.goto('/products');
      await page.getByRole('button', { name: 'Add' }).first().click()
      await expect(page.getByTestId('cart-count')).toHaveText('1');
    });
  });

  test.describe('Payment', () => {
    test('enter card details', async ({ page }) => {
      await page.goto('/checkout');
      await page.getByLabel('Card').fill('4111222233334444');
    });
  });
});
```

Set a custom timeout for a single test block.

▼ 💡 Exercise 2 Solution

code

CODE

```
// Exercise 2 Solution:
test('slow operation with custom timeout', async ({ page }) => {
  test.setTimeout(60000); // 60 second timeout for this test only
  await page.goto('/long-process');
  await page.getByRole('button', { name: 'Start' }).click();
  await expect(page.getByText('Done')).toBeVisible({ timeout: 55000 });
});
```

🚀 Mini-Project

Construct a state isolation wrapper for parallel tests.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
test.describe('Isolated State Tests', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/app');
    // Each test starts fresh due to isolated BrowserContext
  });

  test('user A actions', async ({ page }) => {
    await page.getByLabel('Name').fill('User A');
  });

  test('user B actions', async ({ page }) => {
    // No contamination from User A's state
    await expect(page.getByLabel('Name')).toHaveValue('');
  });
});
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Organize TodoMVC tests into describe blocks with beforeEach navigation:

typescript

TS

```
test.describe("TodoMVC CRUD", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("https://demo.playwright.dev/todomvc");
  });

  test("add todo", async ({ page }) => {
    const input = page.getByPlaceholder("What needs to be done?");
    await input.fill("Organized test");
    await input.press("Enter");
    await expect(page.getByTestId("todo-title")).toHaveText("Organized test");
  });

  test("delete todo", async ({ page }) => { /* ... */ });
});
```

8. Interview Q&A Preparation

Q1: Why should global variables be avoided in describe blocks?

Expected Answer: Since Playwright runs tests in parallel across separate workers, modifying a shared outer variable in one test will corrupt the state in another, causing intermittent failures.

9. Chapter Cheat Sheet

code

CODE

```
test.describe("Group Name", () => { ... });
```

✓ Completed Chapter 17 / 40

43%

PLAYWRIGHT BASICS

ADVANCED PLAYWRIGHT

CHAPTER 18

Auto-Wait Internals

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Deep dive into page actionability checks
- ✓ Expose DOM loading milestones
- ✓ Mitigate hydration delays

CHAPTER 18 • Auto-Wait Internals

1. Why This Matters

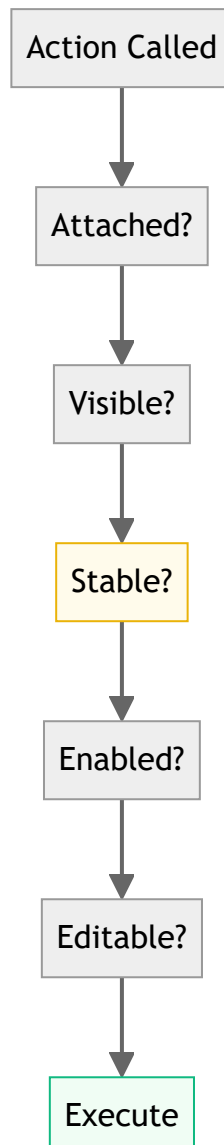
Knowing how auto-waiting handles layouts prevents structural race conditions and reduces the need for manual waits.

2. Conceptual Overview

Auto-waiting parses target elements against detailed accessibility and layout checks. Action calls halt until these checks pass or the action timeout throws an exception.

Hydration delay is the brief pause where an element is painted (visible) but not yet interactive. Web frameworks render initial HTML layout first (server-rendered), then download client-side JavaScript, parse it, and bind the action event handlers. To mitigate this hydration gap, look for indicator styles or wait for hydration attributes (e.g. `data-hydrated="true"`) before clicking.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Wait for animation stability by tracking position stability
test('interact with animating layout', async ({ page }) => {
  const dialogBox = page.locator('.modal-dialog');

  // Explicitly wait for element animation stability milestones
  await dialogBox.waitFor({ state: 'visible' });

  // Perform interaction once stability is guaranteed
  await dialogBox.click();
});
```

4. Best Practices (Do's and Don'ts)

Do

Rely on Playwright auto-waiting instead of adding manual delays.

Check trace files to diagnose actionability check failures.

Don't

Do not wait for load states (networkidle) when checking for single elements.

5. Chapter Summary

Understand DOM milestones: attachment, visibility, stability.

Mitigate React/Vue hydration lags by waiting for target attributes.

Check animation boundaries before triggering pointer coordinate events.

6. Exercises & Mini-Project

Exercises

Simulate an animating element and check how stability checks handle it.

▼ 💡 Exercise 1 Solution

```
code CODE  
  
// Exercise 1 Solution:  
test('handle animating element', async ({ page }) => {  
  await page.goto('/animated-modal');  
  const modal = page.locator('.modal');  
  await modal.waitFor({ state: 'visible' });  
  // Element is now stable and clickable  
  await modal.getByRole('button', { name: 'Confirm' }).click();  
});
```

Configure an action to use an explicit action timeout.

▼ 💡 Exercise 2 Solution

```
code CODE  
  
// Exercise 2 Solution:  
test('action with explicit timeout', async ({ page }) => {  
  await page.goto('/slow-page');  
  const btn = page.getByRole('button', { name: 'Load' });  
  await btn.click({ timeout: 10000 }); // 10s action timeout  
});
```

🚀 Mini-Project

Build a delayed rendering page simulator and write checks for it.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
test('delayed rendering simulator', async ({ page }) => {
  await page.goto('/delayed-content');
  // Content renders after 3 seconds
  const content = page.getByTestId('lazy-content');
  await content.waitFor({ state: 'visible', timeout: 5000 });
  await expect(content).toContainText('Loaded');
  await content.getByRole('button', { name: 'Action' }).click();
});
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Observe auto-wait behavior by toggling a todo and verifying state:

```
typescript TS

test("auto-wait on toggle", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Test auto-wait");
  await input.press("Enter");

  // Click toggle – Playwright auto-waits for the checkbox
  await page.getByRole("checkbox").click();
  // Auto-waits for class change
  await expect(page.getByTestId("todo-item")).toHaveClass(/completed/);
});
```

🎯 8. Interview Q&A Preparation

Q1: How does Playwright determine if an element is stable?

Expected Answer: It compares the bounding box of the element over consecutive animation frames. If the coordinates remain identical, the element is stable.

9. Chapter Cheat Sheet

code

CODE

```
await locator.waitFor({ state: "hidden" });
```

✓ Completed Chapter 18 / 40

45%

ADVANCED PLAYWRIGHT

ADVANCED PLAYWRIGHT

CHAPTER 19

Trace Viewer & Observability

 15 mins Reading Intermediate Level **What You'll Master**

- ✓ Configure trace capture options
- ✓ Inspect console calls and network timelines
- ✓ Diagnose failures using DOM snapshots

CHAPTER 19 • Trace Viewer & Observability

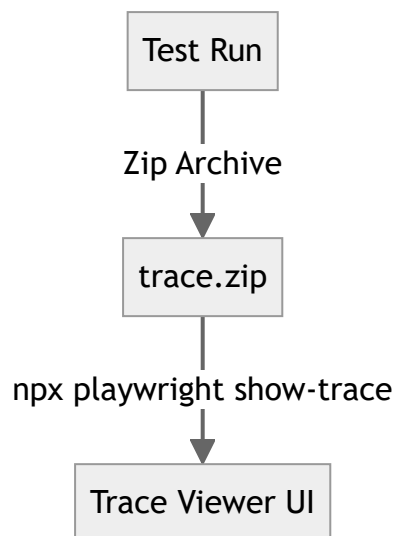
1. Why This Matters

CI runs fail without browser screens. Traces let you step through DOM snapshots at the exact moment of failure to locate bugs.

2. Conceptual Overview

The Trace Viewer captures browser interactions. It records action steps, API calls, browser logs, and DOM snapshots, letting you debug test runs locally or from CI builds.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// playwright.config.ts trace options
import { defineConfig } from '@playwright/test';
export default defineConfig({
  use: {
    trace: 'retain-on-failure',
  }
});
```

4. Best Practices (Do's and Don'ts)

Do

Enable traces on failure in CI/CD.

Inspect network tab streams inside the trace viewer.

● Don't

Do not capture traces for all passing runs, as it slows down executions.

📄 5. Chapter Summary

Capture execution snapshots on retry/failure inside CI pipelines.

Extract network call logs and console prints from trace files.

Inspect DOM snapshots interactively to pinpoint layout bugs.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Run a failing test locally with traces enabled.

▼ 💡 Exercise 1 Solution

```
code CODE
# Exercise 1 Solution:
# 1. Add to playwright.config.ts:
# use: { trace: 'on' }
# 2. Run a failing test:
npx playwright test --headed
# 3. Open the trace:
npx playwright show-trace test-results/test-name/trace.zip
```

Open a generated trace zip file using the Playwright CLI.

▼ 💡 Exercise 2 Solution

```
code CODE  
# Exercise 2 Solution:  
npx playwright show-trace path/to/trace.zip  
# This opens the interactive Trace Viewer in your browser  
# Navigate the timeline, inspect DOM snapshots, and check network cal
```

🚀 Mini-Project

Build a trace parsing script that counts API calls.

▼ 💡 Mini-Project Solution

```
code CODE  
// Mini-Project Solution:  
// Script that reads trace zip and counts API calls  
import * as fs from 'fs';  
// In practice, traces are zip files containing JSON action logs  
// Parse the trace to count network requests:  
console.log('Trace analysis: Count network calls from the trace time1
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Enable trace capture for your TodoMVC tests and inspect a run:

typescript

TS

```
// playwright.config.ts
export default defineConfig({
  use: {
    trace: "retain-on-failure"
  }
});
```

Run a failing test, then open the trace:

code

CODE

```
npx playwright show-trace test-results/.../trace.zip
```

8. Interview Q&A Preparation

Q1: What details are saved inside a Playwright trace archive?

Expected Answer: A zip archive containing screen actions, network requests, console outputs, source maps, and interactive DOM snapshots for every step.

9. Chapter Cheat Sheet

code

CODE

```
npx playwright show-trace path/to/trace.zip
```

✓ Completed Chapter 19 / 40

48%

ADVANCED PLAYWRIGHT

ADVANCED PLAYWRIGHT

CHAPTER 20

Debugging Workflows

 15 mins Reading Intermediate Level **What You'll Master**

- ✓ Step through executions using the Inspector
- ✓ Trace locator hits interactively
- ✓ Run projects selectively

CHAPTER 20 • Debugging Workflows

1. Why This Matters

To write robust test flows, stepping through them interactively allows you to verify that CSS/XPath/ARIA elements are matched before continuing.

2. Conceptual Overview

Debugging in Playwright uses the Playwright Inspector, headed browser projects, and terminal console logs, helping you test selectors and step through actions in real-time.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

// Insert page.pause during dynamic flows
test('run inspector debugging', async ({ page }) => {
  await page.goto('/profile');
  await page.pause(); // Interactive debugger breakpoint
  await page.getByRole('button', { name: 'Edit' }).click();
});
  
```

4. Best Practices (Do's and Don'ts)

Do

Run single tests with the `--debug` flag.

Verify locator expressions inside the inspector console.

Don't

Do not use `console.log` statements as your primary debugging tool.

5. Chapter Summary

Pause executions with `page.pause` to open the inspector.

Debug selector engines directly using browser devtools terminal.

Isolate single failing projects via targeted CLI runs.

6. Exercises & Mini-Project

Exercises

Debug a test using the Playwright Inspector.

▼ Exercise 1 Solution

```
code CODE  
# Exercise 1 Solution:  
# Run with debug flag:  
npx playwright test tests/login.spec.ts --debug  
# The Playwright Inspector opens. Step through each action.  
# Hover over locators in the Inspector to verify they match.
```

Locate and fix a failing button selector in a login spec.

▼ Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('fix failing login button', async ({ page }) => {  
  await page.goto('/login');  
  // Before: page.locator('#btn-login') – brittle ID selector  
  // After: semantic role-based locator  
  const loginBtn = page.getByRole('button', { name: 'Sign In' });  
  await loginBtn.click();  
});
```

Mini-Project

Design a script that automatically pauses tests on assertion failures.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
// Auto-pause on failure using test.afterEach hook:
test.afterEach(async ({ page }, testInfo) => {
  if (testInfo.status !== 'passed') {
    console.log('Test failed! Pausing for inspection...');
    await page.pause(); // Opens Inspector on failure
  }
});
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Debug a failing todo deletion test using the Inspector:

```
typescript TS

test("debug deletion", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Debug me");
  await input.press("Enter");
  await page.pause(); // Inspector opens here!
  // Use Inspector to verify locators before continuing
});
```

🎯 8. Interview Q&A Preparation

Q1: What is the purpose of the `--debug` CLI flag?

Expected Answer: It forces headed execution, sets the test timeout to infinity, and launches the Playwright Inspector to step through the test.

📄 9. Chapter Cheat Sheet

code

CODE

```
npx playwright test --debug
```



✓ Completed Chapter 20 / 40

50%

ADVANCED PLAYWRIGHT

ADVANCED PLAYWRIGHT

CHAPTER 21

Storage State & Auth Strategies

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Save authenticated browser cookies
- ✓ Inject storageState into contexts
- ✓ Manage token expirations

CHAPTER 21 • Storage State & Auth Strategies

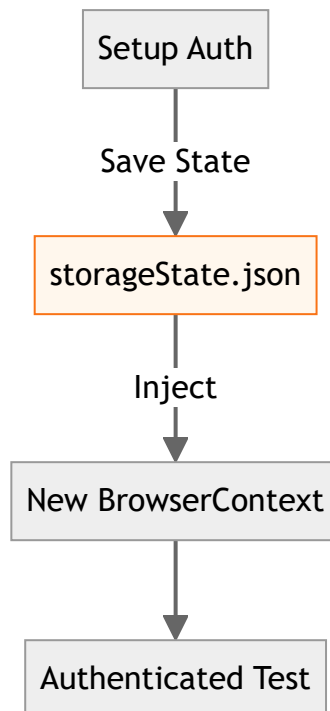
1. Why This Matters

Logging in via the UI before every test slows down runs. Reusing session storage cookies cuts auth setup times.

2. Conceptual Overview

Injecting `storageState` bypasses UI login pages by loading cookies, local storage values, and session data directly into browser contexts during initialization. In production pipelines, session tokens age and expire. To make auth setups robust, tests should evaluate token expiry timestamps parsed from cookies or `localStorage` and trigger an automated re-auth sequence prior to injection.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

import * as fs from 'fs';
import { Browser } from '@playwright/test';

// Setup authenticated context, refreshing tokens dynamically if expired
async function setupAuthenticatedContext(browser: Browser, authFilePath: string) {
  let needsLogin = true;

  if (fs.existsSync(authFilePath)) {
    const authData = JSON.parse(fs.readFileSync(authFilePath, 'utf-8'));
    const expiryCookie = authData.cookies.find((c: any) => c.name === 'session');

    // Check if token has expired compared to the current timestamp
    if (expiryCookie && Date.now() < Number(expiryCookie.value)) {
      needsLogin = false; // Cookie is still valid!
    }
  }

  const options = needsLogin ? {} : { storageState: authFilePath };
  const context = await browser.newContext(options);

  if (needsLogin) {
    const page = await context.newPage();
    await page.goto('/login');
    await page.getByLabel('User').fill('admin');
    await page.getByRole('button', { name: 'Submit' }).click();
    await page.waitForURL('/dashboard');
    await context.storageState({ path: authFilePath });
  }

  return context;
}

```

4. Best Practices (Do's and Don'ts)

Do

Configure globalSetup storage paths in playwright.config.ts.

Clear storage state files when testing user logout scenarios.

Inspect storage JSON files programmatically to evaluate and refresh aged authentication tokens.

● Don't

Do not commit storage state JSON files to source control.

5. Chapter Summary

Save cookie and localStorage states to static JSON files.

Inject storageState into contexts to bypass repetitive UI logins.

Handle token expiration by scheduling proactive login refreshes.

6. Exercises & Mini-Project

🎯 Exercises

Write an auth script that saves a session, then use it in a separate test file.

💡 Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
// File: auth.setup.ts
import { test as setup } from '@playwright/test';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('User').fill('admin');
  await page.getByLabel('Password').fill('secret');
  await page.getByRole('button', { name: 'Login' }).click();
  await page.waitForURL('/dashboard');
  await page.context().storageState({ path: 'auth.json' });
});
```

Verify that browser storage is populated with cookies after setup.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('verify storage populated', async ({ page }) => {  
  await page.goto('/dashboard');  
  const cookies = await page.context().cookies();  
  const sessionCookie = cookies.find(c => c.name === 'session_id');  
  expect(sessionCookie).toBeDefined();  
  console.log('Session cookie:', sessionCookie?.value);  
});
```

🚀 Mini-Project

Build a multi-user storage state manager for a project.

▼ 💡 Mini-Project Solution

```
code CODE  
// Mini-Project Solution:  
const users = ['admin', 'editor', 'viewer'];  
  
for (const role of users) {  
  setup('auth-' + role, async ({ page }) => {  
    await page.goto('/login');  
    await page.getByLabel('User').fill(role);  
    await page.getByLabel('Password').fill(role + '_pass');  
    await page.getByRole('button', { name: 'Login' }).click();  
    await page.context().storageState({ path: role + '.auth.json' });  
  });  
}
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Save and restore todo state using localStorage storage state:

typescript

TS

```
test("save todo state", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Persisted todo");
  await input.press("Enter");
  // Save the browser state (including localStorage with todos)
  await page.context().storageState({ path: "todo-state.json" });
});
```

8. Interview Q&A Preparation

Q1: How does storageState speed up test execution?

Expected Answer: It saves browser sessions (cookies, localStorage) to a file. Subsequent tests load this file to start pre-authenticated, avoiding slow UI login flows.

9. Chapter Cheat Sheet

code

CODE

```
await page.context().storageState({ path: "auth.json" });
```

✓ Completed Chapter 21 / 40

53%

ADVANCED PLAYWRIGHT

ADVANCED PLAYWRIGHT

CHAPTER 22

Network Interception Patterns

 25 mins Reading Intermediate Level **What You'll Master**

- ✓ Mock HTTP APIs using `page.route`
- ✓ Interchange headers and bodies
- ✓ Fulfill mock responses

CHAPTER 22 • Network Interception Patterns

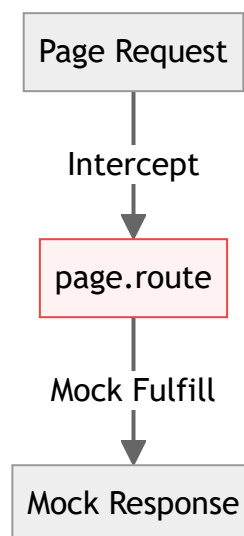
1. Why This Matters

Mocking APIs speeds up execution and isolates layouts, but relying purely on static mocks leads to a major trap: it can mask integration errors, schema mismatches, and contract drift.

2. Conceptual Overview

Playwright supports network routing. Using `page.route`, you can intercept outgoing calls, mock responses, modify headers, and simulate offline scenarios. However, relying too heavily on mocks creates a major drawback: over-mocking can mask critical integration failures, schema drift (where backend API contracts change but tests continue to pass because they use stale mock data), and real environment bugs. To avoid this mocking trap, frameworks should implement contract validation (using tools like Zod to validate live mock shapes against actual schemas) and follow a balanced hybrid approach: use mocks for fast UI state testing, but run real, unmocked end-to-end integration tests nightly. When schema checks fail, report the telemetry payload to a monitoring endpoint for framework engineer diagnostics before failing the test.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
import { test, expect } from '@playwright/test';
import { z } from 'zod';

// Schema representation to prevent contract drift
const UserProfileSchema = z.object({
  id: z.number(),
  name: z.string(),
  email: z.string().email()
});

test('mock api with zod contract verification and logging', async ({ page }) => {
  // Setup route interception first, validating payload during resolution
  await page.route('**/api/profile', async (route) => {
    const mockPayload = { id: 1, name: 'John Doe', email: 'john@drift.com' };
    const validation = UserProfileSchema.safeParse(mockPayload);

    if (!validation.success) {
      // Log schema validation failure telemetry to central monitor
      await page.request.post('https://telemetry.enterprise.com/api/errors', {
        data: {
          service: 'profile-service',
          errors: validation.error.errors,
          timestamp: Date.now()
        }
      });
      await route.abort();
      throw new Error('Schema Drift Detected! Mock data does not match runtime');
    }

    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify(mockPayload)
    });
  });
});
```

```
});  
});
```

🏆 4. Best Practices (Do's and Don'ts)

🟢 Do

Use exact URL matching to avoid intercepting unrelated requests.

Enforce type contract checks (like Zod or JSON Schema) to keep mocks updated.

🔴 Don't

Do not rely on mocked endpoints for all end-to-end integration flows.

Never skip unmocked staging/production tests before major releases.

📄 5. Chapter Summary

Intercept HTTP calls with `page.route` to mock payloads.

Warn against over-mocking to prevent masking integration drifts.

Enforce Zod or JSON schema contract checks to match live APIs.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Write a test that intercepts an API call and returns a mock 500 status code.

🔍 Exercise 1 Solution

```
code CODE  
  
// Exercise 1 Solution:  
test('mock 500 error', async ({ page }) => {  
  await page.route('**/api/data', (route) => {  
    route.fulfill({ status: 500, body: JSON.stringify({ error: 'Serve  
  });  
  await page.goto('/dashboard');  
  await expect(page.getByText('Something went wrong')).toBeVisible();  
});
```

Fulfill an API request with mock JSON data and verify it renders in the UI.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('mock product list', async ({ page }) => {  
  await page.route('**/api/products', (route) => {  
    route.fulfill({  
      status: 200,  
      contentType: 'application/json',  
      body: JSON.stringify([{ name: 'Widget', price: 9.99 }])  
    });  
  });  
  await page.goto('/products');  
  await expect(page.getByText('Widget')).toBeVisible();  
  await expect(page.getByText('$9.99')).toBeVisible();  
});
```

🚀 Mini-Project

Build an offline mode testing simulator for an application.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
test('offline mode simulator', async ({ page }) => {
  // Load page normally first
  await page.goto('/app');
  await expect(page.getByText('Online')).toBeVisible();

  // Block all network requests to simulate offline
  await page.route('**/*', (route) => route.abort());
  await page.reload();
  await expect(page.getByText('Offline')).toBeVisible();
});
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Mock the TodoMVC API to simulate a pre-loaded todo list:

typescript

TS

```
test("mock pre-loaded todos", async ({ page }) => {
  // TodoMVC uses localStorage, so we inject data directly:
  await page.addInitScript(() => {
    const todos = [
      { title: "Mocked Todo 1", completed: false, id: "1" },
      { title: "Mocked Todo 2", completed: true, id: "2" }
    ];
    localStorage.setItem("react-todos", JSON.stringify(todos));
  });
  await page.goto("https://demo.playwright.dev/todomvc");
  await expect(page.getByTestId("todo-title")).toHaveCount(2);
});
```

8. Interview Q&A Preparation

Q1: When should you use `route.abort()` instead of `route.fulfill()`?

Expected Answer: Use `abort()` to simulate network disruptions, connection failures, or to block heavy, non-functional third-party scripts (like analytics).

9. Chapter Cheat Sheet

code

CODE

```
await page.route("**/pattern", route => route.fulfill());
```

✓ Completed Chapter 22 / 40

55%

ADVANCED PLAYWRIGHT

ADVANCED PLAYWRIGHT

CHAPTER 23

Handling IFrames & Tabs

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Scope iframe contexts using `frameLocator`
- ✓ Coordinate multi-tab pages
- ✓ Manage navigation frames

CHAPTER 23 • Handling IFrames & Tabs

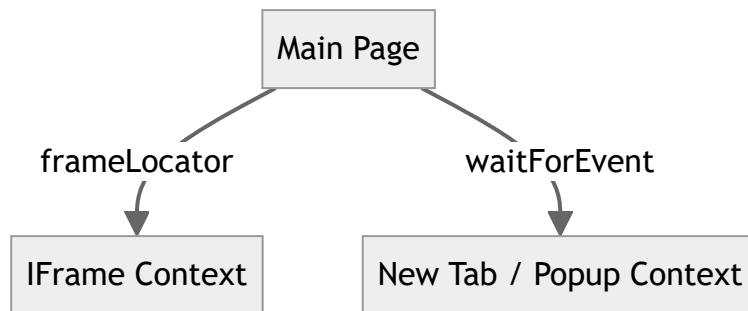
1. Why This Matters

Forgetting to switch contexts when dealing with frames or new tabs causes tests to fail, as page-level actions cannot access nested DOMs.

2. Conceptual Overview

Using `frameLocator` scopes actions inside an iframe. For multi-tab support, register listeners for the popup event on the browser context, returning a new page reference.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Handling switch contexts inside an iframe using frameLocator and popups
test('iframe nested form submission and popup handling', async ({ page }) => {
  await page.goto('/checkout');

  // Locate card element inside iframe scope
  const paymentFrame = page.frameLocator('#payment-iframe');
  await paymentFrame.getByLabel('Card Number').fill('4111222233334444');
  await paymentFrame.getByRole('button', { name: 'Submit Payment' }).click();

  // Handle a popup/tab context event resolution
  const popupPromise = page.context().waitForEvent('popup');
  await page.getByRole('link', { name: 'View Terms' }).click();
  const popupPage = await popupPromise;
  await popupPage.waitForLoadState();
  await expect(popupPage).toHaveTitle('Terms and Conditions');
});
```

4. Best Practices (Do's and Don'ts)

Do

Register the popup event listener before triggering the popup action.

Scope frame locators to unique container IDs.

Don't

Do not try to query nested frames without using frameLocator.

5. Chapter Summary

Scope elements inside nested iframes with frameLocator.

Listen for popup context events before dynamic tab creations.

Switch pages seamlessly using page context event promises.

6. Exercises & Mini-Project

Exercises

Interact with a checkbox inside an iframe.

▼ 💡 Exercise 1 Solution

```
code CODE  
// Exercise 1 Solution:  
test('iframe checkbox interaction', async ({ page }) => {  
  await page.goto('/settings');  
  const frame = page.frameLocator('#preferences-iframe');  
  await frame.getByRole('checkbox', { name: 'Enable Notifications' })  
  await expect(frame.getByRole('checkbox', { name: 'Enable Notificati  
}));
```

Click a link that opens a new tab and assert details on the new page.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('new tab assertion', async ({ page }) => {  
  await page.goto('/links');  
  const popupPromise = page.context().waitForEvent('page');  
  await page.getByRole('link', { name: 'External Link' }).click();  
  const newPage = await popupPromise;  
  await newPage.waitForLoadState();  
  await expect(newPage).toHaveTitle(/External/);  
});
```

🚀 Mini-Project

Build a multi-tab workspace flow manager.

▼ 💡 Mini-Project Solution

```
code CODE
// Mini-Project Solution:
test('multi-tab workspace manager', async ({ page }) => {
  await page.goto('/workspace');
  // Open 3 tabs
  const tabs: Page[] = [];
  for (const name of ['Tab1', 'Tab2', 'Tab3']) {
    const p = page.context().waitForEvent('page');
    await page.getByRole('button', { name }).click();
    tabs.push(await p);
  }
  // Interact with each tab
  for (const tab of tabs) {
    await tab.waitForLoadState();
    console.log('Tab title:', await tab.title());
  }
});
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: TodoMVC does not use iframes or popups, so this is a knowledge checkpoint:

```
code CODE
// TodoMVC is a single-page app with no iframes or popups.
// In real-world apps, you would use:
// page.frameLocator("iframe") – for embedded payment forms
// context.waitForEvent("page") – for links opening new tabs
// Practice these patterns on apps that use them.
```

🎯 8. Interview Q&A Preparation

Q1: Why must you register `context.waitForEvent("popup")` before clicking the link?

Expected Answer: To avoid race conditions. Registering the listener first ensures Playwright catches the popup event the moment the click triggers it.



9. Chapter Cheat Sheet

code

CODE

```
const frame = page.frameLocator("iframe");
```

✓ Completed Chapter 23 / 40

57%

ADVANCED PLAYWRIGHT

ADVANCED PLAYWRIGHT

CHAPTER 24

Emulation & Advanced Browser Settings

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Configure geolocation and timezones
- ✓ Mock locale parameters
- ✓ Emulate mobile layout viewports

CHAPTER 24 • Emulation & Advanced Browser Settings

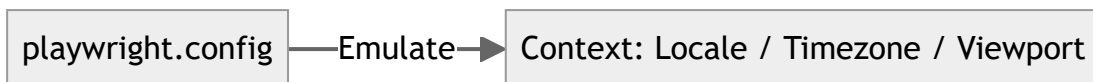
1. Why This Matters

Testing localized content, time zones, or mobile responsive layouts requires browser-level emulation, which Playwright context configurations support.

2. Conceptual Overview

Browser emulation configures location coordinates, timezones, language locales, and responsive viewports, ensuring pages render correctly across devices and regions.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Emulating mobile settings in playwright.config.ts
import { devices } from '@playwright/test';
export default {
  projects: [
    {
      name: 'Mobile Safari',
      use: { ...devices['iPhone 12'] }
    }
  ]
};
```

4. Best Practices (Do's and Don'ts)

Do

Use Playwright built-in devices catalog to configure viewports.

Test locale-specific layouts by updating locale context variables.

Don't

Do not change viewports mid-test unless testing responsive layouts specifically.

5. Chapter Summary

Configure geographic locations, timezones, and language locales.

Match viewports and touch parameters to device profiles.

Validate dark/light themes using color-scheme emulation.

6. Exercises & Mini-Project

Exercises

Write a test that emulates a specific timezone and verify time displays.

▼ Exercise 1 Solution

```
code CODE  
// Exercise 1 Solution:  
test('timezone emulation', async ({ browser }) => {  
  const context = await browser.newContext({ timezoneId: 'America/New  
  const page = await context.newPage();  
  await page.goto('/clock');  
  const time = await page.getByTestId('clock').textContent();  
  console.log('NY Time:', time);  
  await context.close();  
});
```

Emulate mobile screen sizes and test collapsible sidebar menus.

▼ 💡 Exercise 2 Solution

```
code CODE

// Exercise 2 Solution:
test('mobile sidebar collapse', async ({ browser }) => {
  const context = await browser.newContext({ viewport: { width: 375,
  const page = await context.newPage();
  await page.goto('/dashboard');
  await expect(page.getByTestId('sidebar')).not.toBeVisible();
  await page.getByRole('button', { name: 'Menu' }).click();
  await expect(page.getByTestId('sidebar')).toBeVisible();
  await context.close();
});
```

🚀 Mini-Project

Build a localization testing sweep for dynamic applications.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
const locales = ['en-US', 'fr-FR', 'de-DE', 'ja-JP'];
for (const locale of locales) {
  test('locale test: ' + locale, async ({ browser }) => {
    const ctx = await browser.newContext({ locale });
    const page = await ctx.newPage();
    await page.goto('/app');
    const dateText = await page.getByTestId('date-display').textContent;
    console.log(locale + ':', dateText);
    await ctx.close();
  });
}
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Emulate a mobile viewport and verify the TodoMVC responsive layout:

typescript

TS

```
import { devices } from "@playwright/test";

test("mobile TodoMVC layout", async ({ browser }) => {
  const context = await browser.newContext({
    ...devices["iPhone 12"]
  });
  const page = await context.newPage();
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Mobile todo");
  await input.press("Enter");
  await expect(page.getByTestId("todo-title")).toBeVisible();
  await context.close();
});
```

8. Interview Q&A Preparation

Q1: How does Playwright emulate a mobile device context?

Expected Answer: It configures mobile viewports, enables touch events, and updates the user-agent string to match the targeted mobile browser.

9. Chapter Cheat Sheet

code

CODE

```
use: { locale: "fr-FR", timezoneId: "Europe/Paris" }
```

✓ Completed Chapter 24 / 40

60%

ADVANCED PLAYWRIGHT

FRAMEWORK DESIGN

CHAPTER 25

Page Object Model (POM) Design

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Structure Page Objects cleanly
- ✓ Declare locators inside constructors
- ✓ Keep page methods focus to actions

CHAPTER 25 • Page Object Model (POM) Design

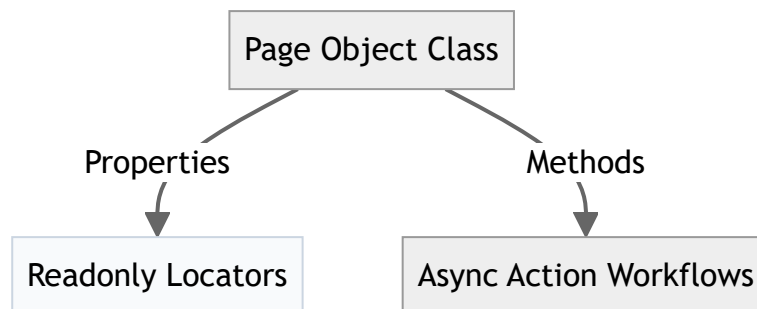
1. Why This Matters

A poor POM structure creates code duplication. Declaring locators in properties and keeping assertions inside tests ensures a maintainable framework.

2. Conceptual Overview

The Page Object Model (POM) abstracts page structures into reusable classes. Declaring locators as properties inside the constructor keeps selectors organized, and page methods handle business actions.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

export class ConfirmationPage {
  readonly page: Page;
  readonly successMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.successMessage = page.getByRole('alert');
  }
}

export class SettingsPage {
  readonly page: Page;
  readonly saveBtn: Locator;

  constructor(page: Page) {
    this.page = page;
    this.saveBtn = page.getByRole('button', { name: 'Save Changes' });
  }

  // Action methods return target POM pages to chain flows cleanly
  async applySettings(): Promise<ConfirmationPage> {
    await this.saveBtn.click();
    return new ConfirmationPage(this.page);
  }
}

```

🏆 4. Best Practices (Do's and Don'ts)



Do

Declare locators as readonly class properties.

Ensure page actions return locators or state variables for tests to assert.



Don't

Do not write assertions (expect) inside Page Object methods.

Do not declare locators as class methods.

📄 5. Chapter Summary

Declare readonly locator properties in the page constructors.

Keep test assertions decoupled from POM action methods.

Return child page references or locators to chain flows cleanly.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Build a Page Object class for a registration flow.

▼ 💡 Exercise 1 Solution

```
code CODE

// Exercise 1 Solution:
export class RegisterPage {
  readonly page: Page;
  readonly nameInput: Locator;
  readonly emailInput: Locator;
  readonly submitBtn: Locator;

  constructor(page: Page) {
    this.page = page;
    this.nameInput = page.getByLabel('Full Name');
    this.emailInput = page.getByLabel('Email Address');
    this.submitBtn = page.getByRole('button', { name: 'Register' });
  }

  async registerUser(name: string, email: string) {
    await this.nameInput.fill(name);
    await this.emailInput.fill(email);
    await this.submitBtn.click();
  }
}
```

Refactor a script containing direct page queries to use a Page Object model instead.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
test('login using Page Object model', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.navigate();
  await loginPage.login('user', 'pass');
  await expect(page).toHaveURL(/dashboard/);
});
```

🚀 Mini-Project

Construct a unified POM library for an application checkout funnel.

▼ 💡 Mini-Project Solution

```
code CODE
// Mini-Project Solution:
class CheckoutPage {
  readonly page: Page;
  constructor(page: Page) { this.page = page; }
  async fillShipping() { /* ... */ }
  async pay(): Promise<ConfirmationPage> {
    await this.page.click('text=Pay');
    return new ConfirmationPage(this.page);
  }
}
```

📁 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Build a Page Object class for the TodoMVC page:

typescript

TS

```
// pages/ToDoPage.ts
import { Page, Locator, expect } from "@playwright/test";

export class ToDoPage {
  readonly page: Page;
  readonly input: Locator;
  readonly todoItems: Locator;

  constructor(page: Page) {
    this.page = page;
    this.input = page.getByPlaceholder("What needs to be done?");
    this.todoItems = page.getByTestId("todo-title");
  }

  async navigate() {
    await this.page.goto("https://demo.playwright.dev/todomvc");
  }

  async addTodo(text: string) {
    await this.input.fill(text);
    await this.input.press("Enter");
  }
}
```

8. Interview Q&A Preparation

Q1: Why is it an anti-pattern to include assertions inside Page Objects?

Expected Answer: It couples the page layout with test assertions. If the page is reused in negative test cases (e.g. testing input errors), internal assertions will cause the test to fail.

9. Chapter Cheat Sheet

code

CODE

```
class MyPage { constructor(page: Page) { ... } }
```

✓ Completed Chapter 25 / 40

FRAMEWORK DESIGN

63%

FRAMEWORK DESIGN

CHAPTER 26

Reusable UI Component Encapsulation

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Identify modular widgets
- ✓ Scope locators to component roots
- ✓ Compose Page Objects

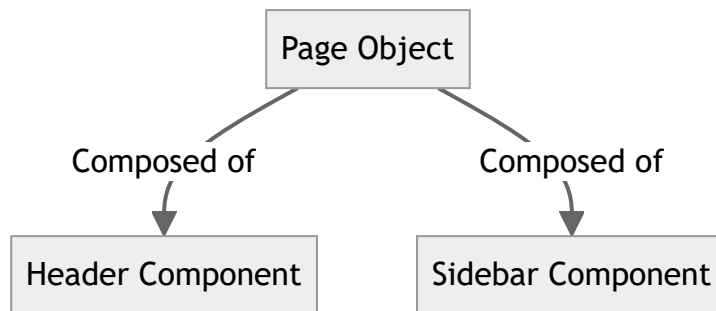
CHAPTER 26 • Reusable UI Component Encapsulation

1. Why This Matters

Duplicate selector definitions for common layouts (like navbars or tables) cause high code churn when headers change. Encapsulating them into components solves this.

2. Conceptual Overview

Encapsulating page widgets (headers, sidebars, lists) into standalone component classes allows you to reuse them across multiple pages. Composition is preferred over inheritance. Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

export class NavbarComponent {
  readonly root: Locator;
  readonly profileLink: Locator;

  constructor(root: Locator) {
    this.root = root;
    this.profileLink = root.getByRole('link', { name: 'Profile' });
  }

  async clickProfile() {
    await this.profileLink.click();
  }
}

export class DashboardPage {
  readonly page: Page;
  readonly navbar: NavbarComponent;

  constructor(page: Page) {
    this.page = page;
    this.navbar = new NavbarComponent(page.getByRole('navigation', { name: 'Ma:
  }
}

```

🏆 4. Best Practices (Do's and Don'ts)



Do

Scope all component selectors to a parent root locator.

Instantiate component sub-classes inside Page Object constructors.



Don't

Avoid deep class inheritance extensions (e.g. extends BasePage).

📄 5. Chapter Summary

Abstract repeating UI controls (navbars, cards) into component classes.

Scope component selectors strictly to a parent root locator.

Favor composition over inheritance to model layout relationships.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Encapsulate a table row layout into a TableRow component.

▼ 💡 Exercise 1 Solution

```
code CODE

// Exercise 1 Solution:
export class TableRowComponent {
  readonly root: Locator;
  readonly editBtn: Locator;
  readonly deleteBtn: Locator;

  constructor(root: Locator) {
    this.root = root;
    this.editBtn = root.getByRole('button', { name: 'Edit' });
    this.deleteBtn = root.getByRole('button', { name: 'Delete' });
  }
}
```

Build a Page Object composed of separate Sidebar and Header components.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
export class DashboardPage {
  readonly sidebar: SidebarComponent;
  readonly header: HeaderComponent;

  constructor(page: Page) {
    this.sidebar = new SidebarComponent(page.locator('aside'));
    this.header = new HeaderComponent(page.locator('header'));
  }
}
```

🚀 Mini-Project

Build a component dashboard library with modular widget classes.

▼ 💡 Mini-Project Solution

```
code CODE
// Mini-Project Solution:
export class DashboardLibrary {
  readonly statsWidget: StatsWidgetComponent;
  readonly chartWidget: ChartWidgetComponent;
  constructor(page: Page) {
    this.statsWidget = new StatsWidgetComponent(page.locator('.stats'));
    this.chartWidget = new ChartWidgetComponent(page.locator('.charts'));
  }
}
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Extract a `TodoItemComponent` for individual todo rows to isolate item elements:

typescript

TS

```
// components/TodoItemComponent.ts
import { Locator } from "@playwright/test";

export class TodoItemComponent {
  readonly root: Locator;
  readonly label: Locator;
  readonly toggleCheckbox: Locator;
  readonly destroyBtn: Locator;

  constructor(root: Locator) {
    this.root = root;
    this.label = root.getByTestId("todo-title");
    this.toggleCheckbox = root.getByRole("checkbox");
    this.destroyBtn = root.getByRole("button", { name: "Delete" });
  }

  async toggle() {
    await this.toggleCheckbox.click();
  }

  async delete() {
    await this.root.hover();
    await this.destroyBtn.click();
  }
}
```

8. Interview Q&A Preparation

Q1: Why should component class selectors be scoped to a parent root locator?

Expected Answer: Scoping prevents locator duplication errors. It ensures Playwright searches only within that widget region, even if other pages have matching elements.

9. Chapter Cheat Sheet

code

CODE

```
this.root = page.locator("selector");
```



✓ Completed Chapter 26 / 40

65%

FRAMEWORK DESIGN

FRAMEWORK DESIGN

CHAPTER 27

Fixtures & Dependency Injection

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Configure custom fixtures using `test.extend`
- ✓ Auto-inject Page Objects
- ✓ Manage fixture lifecycles

CHAPTER 27 • Fixtures & Dependency Injection

1. Why This Matters

Instantiating page classes manually inside every `beforeEach` hook is repetitive. Custom fixtures automate this setup, keeping test specs clean.

2. Conceptual Overview

Playwright fixtures provide dependency injection. By extending `test`, we configure page objects that are automatically instantiated and injected into test arguments.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

import { test as base } from '@playwright/test';
import { HomePage } from '../pages/HomePage';
import { SettingsPage } from '../pages/SettingsPage';

export const test = base.extend<{ homePage: HomePage; settingsPage: SettingsPage }>({
  homePage: async ({ page }, use) => {
    await use(new HomePage(page));
  },
  settingsPage: async ({ page }, use) => {
    await use(new SettingsPage(page));
  }
});
  
```

4. Best Practices (Do's and Don'ts)

Do

Use extended fixtures to inject page objects into tests.

Define test-level cleanup steps after the `await use()` statement.

Don't

Do not call manual page class instantiations inside spec test files.

5. Chapter Summary

Use `test.extend` to configure automatic POM instances.

Run fixture setups and teardowns automatically around tests.

Define worker-level fixtures to share heavy backend clients.

6. Exercises & Mini-Project

Exercises

Extend a base test setup to inject a custom settings page object.

Exercise 1 Solution

```
code CODE

// Exercise 1 Solution:
import { test as base } from './fixtures';
import { SettingsPage } from './pages/SettingsPage';

export const test = base.extend<{ settingsPage: SettingsPage }>({
  settingsPage: async ({ page }, use) => {
    await use(new SettingsPage(page));
  }
});
```

Implement an authenticated page fixture that handles login setups automatically.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
export const test = base.extend<{ authenticatedPage: Page }>({
  authenticatedPage: async ({ browser }, use) => {
    const context = await browser.newContext({ storageState: 'auth.js
    const page = await context.newPage();
    await use(page);
    await context.close();
  }
});
```

🚀 Mini-Project

Build a fixture composition layer for an enterprise test suite.

▼ 💡 Mini-Project Solution

```
code CODE
// Mini-Project Solution:
// Composition of fixtures:
export const test = base.extend<{ adminPage: AdminPage; userPage: Use
  adminPage: async ({ page }, use) => {
    await use(new AdminPage(page));
  },
  userPage: async ({ page }, use) => {
    await use(new UserPage(page));
  }
});
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Create a custom fixture that injects the `TodoPage` instance automatically:

typescript

TS

```
// fixtures/todo-fixture.ts
import { test as base } from "@playwright/test";
import { TodoPage } from "../pages/TodoPage";

export const test = base.extend<{ todoPage: TodoPage }>({
  todoPage: async ({ page }, use) => {
    const todoPage = new TodoPage(page);
    await todoPage.navigate();
    await use(todoPage);
  }
});

// Then write tests cleanly:
// test("test using POM fixture", async ({ todoPage }) => {
//   await todoPage.addTo("Auto-injected");
// });
```

8. Interview Q&A Preparation

Q1: What is the lifecycle of a test-scoped fixture?

Expected Answer: It is instantiated before the test begins, injected into the test arguments, and any code after the `await use()` call executes as cleanup when the test finishes.

9. Chapter Cheat Sheet

code

CODE

```
export const test = base.extend<{
  homePage: HomePage;
  settingsPage: SettingsPage;
}>({
  homePage: async ({ page }, use) => { await use(new HomePage(page)); },
  settingsPage: async ({ page }, use) => { await use(new SettingsPage(page)); }
});
```

✓ Completed Chapter 27 / 40

FRAMEWORK DESIGN

68%

FRAMEWORK DESIGN

CHAPTER 28

4-Layer Architecture Topology

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Organize automation projects in 4 levels
- ✓ Define boundaries between layers
- ✓ Map mock data schemas

CHAPTER 28 • 4-Layer Architecture Topology

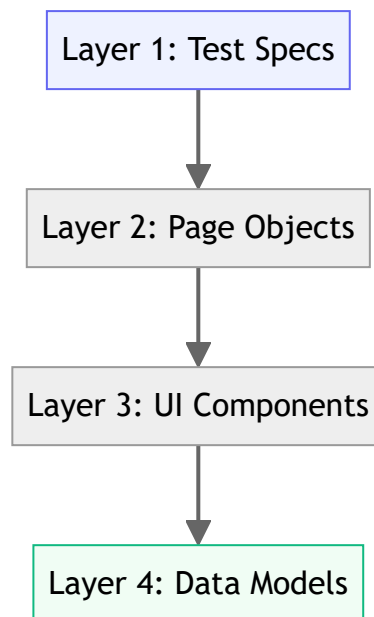
1. Why This Matters

Unstructured frameworks quickly become hard to maintain. A 4-layer architecture isolates test files, page flows, widgets, and data schemas.

2. Conceptual Overview

The 4-Layer Page Object Architecture separates files by responsibility. This structure isolates assertions, page navigation, reusable widgets, and typing schemas, ensuring a clean codebase.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// =====
// LAYER 4: Entity Schemas (File: types/UserPayload.ts)
// =====
export interface UserPayload {
  name: string;
  email: string;
}

// =====
// LAYER 3: Components (File: components/ProfileForm.ts)
// =====
import { Locator } from '@playwright/test';
export class ProfileFormComponent {
  readonly root: Locator;
  constructor(root: Locator) { this.root = root; }
  async fillForm(data: UserPayload) {
    await this.root.getByLabel('Name').fill(data.name);
    await this.root.getByLabel('Email').fill(data.email);
  }
}

// =====
// LAYER 2: Pages (File: pages/ProfilePage.ts)
// =====
import { Page } from '@playwright/test';
export class ProfilePage {
  readonly page: Page;
  readonly profileForm: ProfileFormComponent;
  constructor(page: Page) {
    this.page = page;
    this.profileForm = new ProfileFormComponent(page.locator('form#profile'));
  }
}

// =====
// LAYER 1: Specs (File: specs/profile.spec.ts)
```

```
// =====
import { test } from '@playwright/test';
test('validate profile form update', async ({ page }) => {
  const profilePage = new ProfilePage(page);
  const data: UserPayload = { name: 'Alice', email: 'alice@test.com' };
  await page.goto('/profile');
  await profilePage.profileForm.fillForm(data);
});
```

🏆 4. Best Practices (Do's and Don'ts)

🟢 Do

Isolate specs, pages, components, and data schemas in separate folders.

Verify that data schema definitions (Layer 4) remain clean.

🔴 Don't

Do not mix test spec assertions (Layer 1) inside page classes (Layer 2).

📄 5. Chapter Summary

Isolate specs, pages, components, and typing layers.

Maintain clean boundaries: specs assert, components locate.

Type data payloads with TypeScript interfaces to avoid syntax errors.

🏋️ 6. Exercises & Mini-Project

🎯 Exercises

Restructure a basic project directory into a 4-layer folder structure.

💡 Exercise 1 Solution

code

CODE

```
# Exercise 1 Solution:
# Restructure your files as follows:
# - Move *.spec.ts to /specs
# - Move page classes to /pages
# - Move reusable widgets to /components
# - Move interfaces/types to /types
```

Map static API JSON models into TypeScript interfaces.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
export interface UserJsonModel {  
  id: string;  
  name: string;  
  preferences: {  
    theme: 'light' | 'dark';  
  };  
};  
}
```

🚀 Mini-Project

Restructure a legacy repository using 4-layer design guidelines.

▼ 💡 Mini-Project Solution

```
code CODE  
// Mini-Project Solution:  
// Organized structure setup with imports mapping boundaries:  
import { test } from '../fixtures/todo-fixture';  
import { UserPayload } from '../types/UserPayload';  
// Test specs import Layer 2 page/fixture layers correctly.
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Organize the TodoMVC automation project into a 4-layer structure:

code

CODE

```

todomvc-automation/
├── specs/
│   └── todo.spec.ts          (Layer 1: Test Specs)
├── pages/
│   └── TodoPage.ts          (Layer 2: Page Objects)
├── components/
│   └── TodoItemComponent.ts (Layer 3: UI Components)
├── types/
│   └── TodoItem.ts          (Layer 4: Data Models/Entities)
├── package.json
└── playwright.config.ts

```

8. Interview Q&A Preparation

Q1: How does a 4-layer architecture simplify design updates?

Expected Answer: By separating concerns. If a table widget changes, you only update the Component class in Layer 3; Layer 1 (Specs) and Layer 2 (Pages) remain untouched.

9. Chapter Cheat Sheet

code

CODE

```
Folder structure: specs/, pages/, components/, types/
```

✓ Completed Chapter 28 / 40

70%

FRAMEWORK DESIGN

CI/CD & ENTERPRISE

CHAPTER 29

Playwright Test Runner Internals

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Explain parallel execution models
- ✓ Configure worker allocations
- ✓ Understand CPU thread setups

CHAPTER 29 • Playwright Test Runner Internals

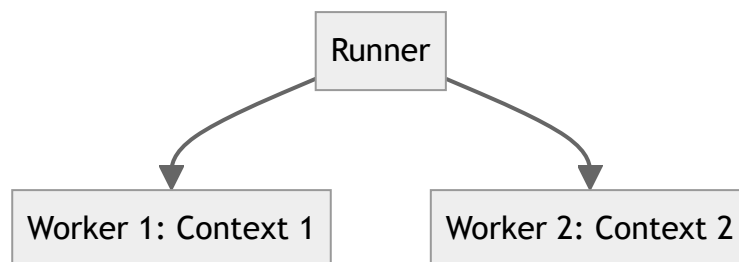
1. Why This Matters

Misconfiguring parallel worker limits can freeze system resources or cause dynamic test collisions. Optimizing workers keeps runs fast and stable.

2. Conceptual Overview

The Playwright test runner runs tests in parallel across isolated worker processes. Each worker starts its own browser context, ensuring tests do not interfere with one another.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// config workers allocations sample
export default {
  workers: process.env.CI ? 2 : undefined,
  fullyParallel: true
};
```

4. Best Practices (Do's and Don'ts)

Do

Enable `fullyParallel` option to run tests concurrently.

Limit worker count on CI servers to match available CPU cores.

Don't

Do not share global states across test files when running in parallel.

5. Chapter Summary

Spawn independent worker processes to achieve concurrency.

Run tests `fullyParallel` with independent browser contexts.

Tune worker allocations to match physical CPU core counts.

6. Exercises & Mini-Project

Exercises

Configure test runs to use a specific number of workers from the CLI.

▼ Exercise 1 Solution

code

CODE

```
# Exercise 1 Solution:
npx playwright test --workers=2
```

Measure the speed difference between running tests sequentially vs. in parallel.

▼ Exercise 2 Solution

code

CODE

```
# Exercise 2 Solution:
# Run tests sequentially:
npx playwright test --workers=1
# Run tests in parallel:
npx playwright test --workers=4
# Compare the durations printed in the run summaries.
```

Mini-Project

Build a monitor script that tracks CPU load during test execution.

▼ 💡 Mini-Project Solution

```
code CODE  
  
// Mini-Project Solution:  
// A helper process monitor reading os.cpus() load  
import * as os from 'os';  
console.log('CPU Load:', os.cpus());
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Configure worker allocations in your `playwright.config.ts` for TodoMVC tests:

```
typescript TS  
  
// playwright.config.ts  
import { defineConfig } from "@playwright/test";  
  
export default defineConfig({  
  workers: process.env.CI ? 2 : undefined,  
  fullyParallel: true,  
  use: {  
    baseURL: "https://demo.playwright.dev/todomvc"  
  }  
});
```

🎯 8. Interview Q&A Preparation

Q1: How does Playwright isolate parallel test runs?

Expected Answer: By running each test in a separate worker process with its own BrowserContext, isolating cookies, storage, and sessions.

📄 9. Chapter Cheat Sheet

code

CODE

```
npx playwright test --workers=4
```



✓ Completed Chapter 29 / 40

73%

CI/CD & ENTERPRISE

CI/CD & ENTERPRISE

CHAPTER 30

Flaky Test Troubleshooting

 20 mins Reading Advanced Level

What You'll Master

- ✓ Identify flaky test causes
- ✓ Examine execution traces
- ✓ Implement retry strategies

CHAPTER 30 • Flaky Test Troubleshooting

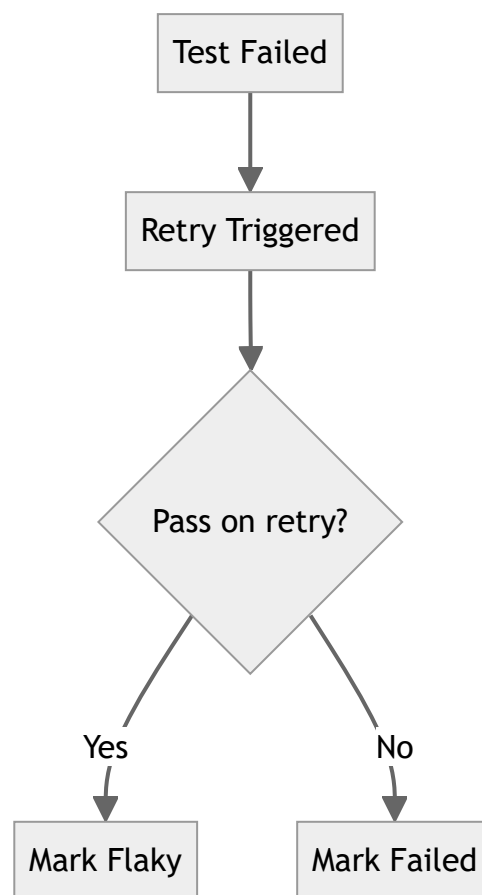
1. Why This Matters

Flaky tests slow down pipelines and erode trust in automation. Finding and fixing flakes keeps CI builds reliable.

2. Conceptual Overview

Flakiness is often caused by race conditions, slow networks, or dynamic DOM states. Debugging flakes requires inspecting trace snapshots, checking API responses, and refining wait conditions. The chapter strictly forbids using `page.waitForTimeout` to patch flakiness. When testing asynchronous non-DOM systems, such as background worker operations, message brokers, or WebSockets, tests should use event listeners (`page.waitForEvent`) or poll status tables using `expect.poll` to wait for completion.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// 1. Wait for WebSocket message packet response instead of sleeping
const messagePromise = page.waitForEvent('websocket');
await page.getByRole('button', { name: 'Submit Query' }).click();
const ws = await messagePromise;

// 2. Poll a state database backend API status
await expect.poll(async () => {
  return await checkBackgroundJob();
}).toBe('COMPLETED');
```

🏆 4. Best Practices (Do's and Don'ts)



Do

Inspect trace files to identify gaps in timelines.

Ensure test assertions wait for dynamic elements to render.



Don't

Do not use arbitrary timeouts (page.waitForTimeout) to address flakiness.

📄 5. Chapter Summary

Debug race conditions by aligning assertions with state changes.

Set up automatic retries for CI builds to handle infrastructure lag.

Parse HTML results to isolate environment flakes from bugs.

🏋️ 6. Exercises & Mini-Project



Exercises

Find the root cause of an intermittent assertion failure by analyzing its trace.

▼ 💡 Exercise 1 Solution

```
code CODE  
// Exercise 1 Solution:  
// 1. Locate the test run report or trace viewer logs  
// 2. Identify timelines with long gaps between actions (e.g. 5000ms  
// 3. Pinpoint the missing state transitions or unawaited elements.
```

Simulate a slow network response and verify the test handles it without failing.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('slow API network mock', async ({ page }) => {  
  await page.route('**/api/data', async (route) => {  
    await new Promise(resolve => setTimeout(resolve, 3000)); // Delay  
    await route.fulfill({ status: 200, body: '{}' });  
  });  
  await page.goto('/dashboard');  
  await expect(page.getByText('Data Loaded')).toBeVisible({ timeout:  
});
```

🚀 Mini-Project

Create a script that parses test reports to find and tag flaky specs.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
import * as fs from 'fs';
interface TestResult { title: string; status: 'passed' | 'failed' | '
function tagFlakes(results: TestResult[]) {
  return results.filter(r => r.status === 'flaky').map(r => r.title);
}
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Diagnose a flaky toggle test by checking for element stability gates:

typescript

TS

```
test("stable toggle check", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Flake fix");
  await input.press("Enter");

  const checkbox = page.getByRole("checkbox");
  // Ensure element is fully stable and visible before check
  await expect(checkbox).toBeVisible();
  await checkbox.click();
  await expect(page.getByTestId("todo-item")).toHaveClass(/completed/);
});
```

🎯 8. Interview Q&A Preparation

Q1: How do you fix a flaky check caused by dynamic data loading?

Expected Answer: Replace hardcoded delays with Web-First assertions (like `expect().toBeVisible()`) or wait explicitly for network events.

📄 9. Chapter Cheat Sheet

code

CODE

```
npx playwright test --retries=2
```

✓ Completed Chapter 30 / 40

75%

CI/CD & ENTERPRISE

CI/CD & ENTERPRISE

CHAPTER 31

Onboarding & Transition Guides

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Establish coding standards
- ✓ Migrate legacy test suites
- ✓ Configure AST linter rules

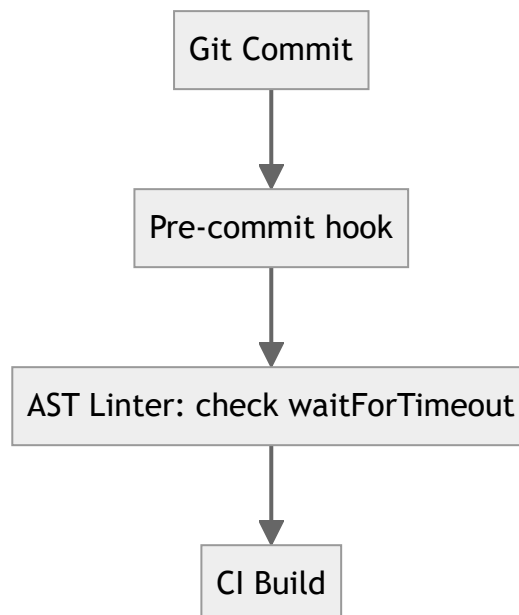
CHAPTER 31 • Onboarding & Transition Guides

1. Why This Matters

Onboarding new developers to a unified codebase prevents style deviations. Automated AST checks enforce these coding rules by inspecting the compiler syntax trees.

2. Conceptual Overview

When a compiler or linter parses code, it builds an Abstract Syntax Tree (AST). By traversing this tree programmatically, we can detect and block unsafe anti-patterns—such as calling `page.waitForTimeout(5000)`—without relying on fragile regex search patterns. This chapter details Cypress-to-Playwright migration maps, framework standards, and custom linter setups. To enforce quality gates, the pre-commit checker must execute a `process.exit(1)` code if any issues are flagged, forcing the git pre-commit hook to abort. Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

import * as ts from 'typescript';
import * as fs from 'fs';
import * as path from 'path';

// Programmatic AST Checker using TypeScript Compiler API
export function checkFileForBannedMethods(filePath: string): boolean {
  const sourceCode = fs.readFileSync(filePath, 'utf-8');
  const sourceFile = ts.createSourceFile(filePath, sourceCode, ts.ScriptTarget.Latest);
  let isClean = true;

  function visit(node: ts.Node) {
    if (ts.isCallExpression(node)) {
      const expression = node.expression;
      let methodName = '';
      if (ts.isPropertyAccessExpression(expression)) {
        methodName = expression.name.text;
      }

      // Enforce zero tolerance for waitForTimeout
      if (methodName === 'waitForTimeout') {
        const { line } = sourceFile.getLineAndCharacterOfPosition(node.getStart());
        console.error('✖ AST Error at ' + filePath + ':' + (line + 1) + ' - Banned method: ' + methodName);
        isClean = false;
      }
    }
    ts.forEachChild(node, visit);
  }

  visit(sourceFile);
  return isClean;
}

// Programmatic directory scan sweep for pre-commit quality checks
function getTestFiles(dir: string): string[] {
  let results: string[] = [];
  const list = fs.readdirSync(dir);

```

```

list.forEach((file) => {
  const fullPath = path.join(dir, file);
  const stat = fs.statSync(fullPath);
  if (stat && stat.isDirectory()) {
    results = results.concat(getTestFiles(fullPath));
  } else if (file.endsWith('.spec.ts')) {
    results.push(fullPath);
  }
});
return results;
}

// Running quality gate execution wrapper
const targetDir = path.join(__dirname, '../tests');
const files = getTestFiles(targetDir);
let codebaseClean = true;
for (const file of files) {
  if (!checkFileForBannedMethods(file)) {
    codebaseClean = false;
  }
}

if (!codebaseClean) {
  console.error('❌ Code quality validation failed. Pre-commit aborted.');
```

process.exit(1); // Exits the process with code 1 to block hook commits!

```

}

```

4. Best Practices (Do's and Don'ts)

Do

Maintain clear transition guides for engineers moving from other frameworks.

Enforce coding standards automatically using custom AST linter checks.

Don't

Avoid manually checking for styling rule violations during code reviews.

5. Chapter Summary

Use transition maps for Cypress or Selenium developer migration.

Implement AST checkers to block banned API methods dynamically.

Lint codebase formatting before commit hooks trigger pipelines.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Write an AST script that flags any occurrence of `page.waitForTimeout`.

▼ 💡 Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
import * as ts from 'typescript';
// AST parser checking for banned wait statements:
// Search for CallExpression where expression.name.text === 'waitForT
// Throw compile/AST exit error code: process.exit(1)
```

Translate a Selenium Java test block into a Playwright TypeScript spec.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
// Cypress: cy.visit('/login'); cy.get('#user').type('admin'); cy.con
// Playwright:
await page.goto('/login');
await page.getByLabel('Username').fill('admin');
await page.getByRole('button', { name: 'Submit' }).click();
```

🚀 Mini-Project

Build a Cypress-to-Playwright code translation helper.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
// Simple text replacer Cypress helper:
function translateCypressToPlaywright(cyCode: string): string {
  return cyCode
    .replace(/cy\.visit\(((\[^\)]+)\))/g, 'await page.goto($1)')
    .replace(/cy\.get\(((\[^\)]+)\))\.type\(((\[^\)]+)\))/g, 'await page.locate($1).type($2)')
}
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Build an AST checker that prevents using `page.waitForTimeout` inside your TodoMVC automation project:

typescript

TS

```
// scripts/ast-guard.ts
import * as ts from "typescript";
import * as fs from "fs";
// Set up programmatic AST checks on spec files to block waitForTimeout
// If any violation is found, exit process with process.exit(1) to block git commit
```

🎯 8. Interview Q&A Preparation

Q1: Why are automated quality gates preferred over manual code reviews for code styles?

Expected Answer: Automated gates are consistent, run instantly on commit, and prevent style arguments, leaving code reviews focused on architecture and logic.

📄 9. Chapter Cheat Sheet

code

CODE

```
npm run lint // run coding standards lint
```



✓ Completed Chapter 31 / 40

78%

CI/CD & ENTERPRISE

CI/CD & ENTERPRISE

CHAPTER 32

Test Data Management

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Use data factories
- ✓ Avoid static JSON data hazards
- ✓ Implement database seeding

CHAPTER 32 • Test Data Management

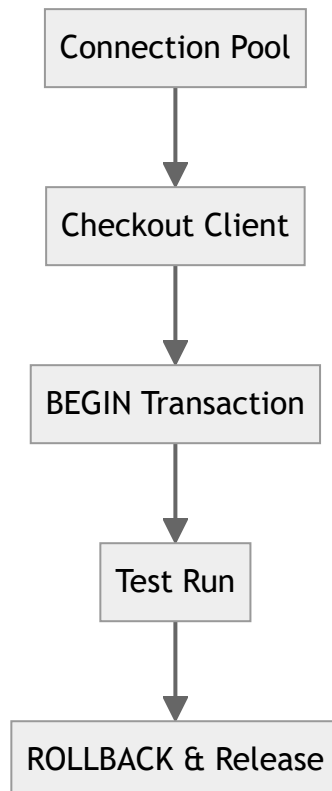
1. Why This Matters

Sharing static database records in parallel test runs leads to locks and validation collision. Transactional seeding isolates runs.

2. Conceptual Overview

Managing test data requires using dynamic data factories to generate unique parameters for each test, ensuring tests can run in parallel without conflicts. For relational data, direct database connections should be checked out from a client connection Pool. By connecting, starting a SQL transaction (BEGIN) before the test runs, and executing rollbacks (ROLLBACK) inside a finally block, tests preserve database state isolation and release connections securely back to the pool, preventing connection leaks, deadlocks, and pool exhaustion.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

import { test as base } from '@playwright/test';
import { Pool } from 'pg';

// Setup connection Pool with limits
const pool = new Pool({
  connectionString: process.env.DATABASE_URL || 'postgresql://postgres:postgres@localhost:5432/postgres',
  max: 20, // Avoid connection exhaustion
  idleTimeoutMillis: 30000
});

// Setup database connection and transaction isolation fixture
export const dbIsolatedTest = base.extend<{ dbClient: any }>({
  dbClient: async ({}, use) => {
    // Checkout client from connection pool
    const client = await pool.connect();

    try {
      // Start transaction isolation layer
      await client.query('BEGIN');

      // Pass client reference to the test spec
      await use(client);
    } finally {
      // Guaranteed database transaction rollback on success OR crash
      try {
        await client.query('ROLLBACK');
      } catch (rollbackErr) {
        console.error('Rollback failed:', rollbackErr);
      } finally {
        // Safely release connection client back to the pool to prevent leaks
        client.release();
      }
    }
  }
});

```

🏆 4. Best Practices (Do's and Don'ts)

🟢 Do

Use data factories to generate unique payloads dynamically.

Roll back database seeding operations using transaction blocks.

Always release database clients inside finally blocks to prevent pool exhaustion.

🔴 Don't

Do not rely on static JSON user records for tests that mutate data.

📄 5. Chapter Summary

Generate unique test parameters with dynamic data factories.

Seed database records via DB connection scripts inside hooks.

Rollback database transactions to clean state without teardowns.

🏋️ 6. Exercises & Mini-Project

🎯 Exercises

Build a data factory helper that returns unique product entries.

💡 Exercise 1 Solution

```
code CODE

// Exercise 1 Solution:
export class ProductFactory {
  static create(overrides = {}) {
    return {
      id: Math.floor(Math.random() * 1000),
      name: 'Product-' + Date.now(),
      price: parseFloat((Math.random() * 100).toFixed(2)),
      ...overrides
    };
  }
}
```

Write a setup hook that seeds a user account via an API call.

▼ 💡 Exercise 2 Solution

```
code CODE

// Exercise 2 Solution:
test.beforeEach(async ({ request }) => {
  const res = await request.post('/api/users/seed', {
    data: { username: 'testuser', role: 'admin' }
  });
  expect(res.status()).toBe(201);
});
```

🚀 Mini-Project

Build a dynamic billing data generator for enterprise testing.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
export class BillingFactory {
  static createCardPayload(type: 'visa' | 'mastercard') {
    return {
      number: type === 'visa' ? '4111222233334444' : '5100000000000000',
      cvv: '123',
      name: 'Cardholder Name'
    };
  }
}
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Build a test data factory that generates random todo titles for your tests:

typescript

TS

```
// utils/DataFactory.ts
export class DataFactory {
  static generateTodoTitle(): string {
    return "Todo-" + Math.random().toString(36).substring(2, 7);
  }
}

// Inside your test spec:
// await page.fill(input, DataFactory.generateTodoTitle());
```

8. Interview Q&A Preparation

Q1: What is the hazard of static JSON test data in parallel runs?

Expected Answer: If multiple tests use the same static account details concurrently, actions in one test (like updating a password) will log out or fail other tests using that account.

9. Chapter Cheat Sheet

code

CODE

```
const user = DataFactory.makeUser();
```

✓ Completed Chapter 32 / 40

80%

CI/CD & ENTERPRISE

API TESTING

CHAPTER 33

APIRequestContext & HTTP Calls

 20 mins Reading Intermediate Level **What You'll Master**

- ✓ Perform API requests using the request fixture
- ✓ Validate API response states
- ✓ Set up token headers

CHAPTER 33 • APIRequestContext & HTTP Calls

1. Why This Matters

API testing is faster than UI testing. Using direct API requests validates service layers and speeds up test setups.

2. Conceptual Overview

Using Playwright APIRequestContext, you can trigger HTTP calls directly inside your tests. This is useful for validating backend APIs and setting up test states.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
test('verify API product creation', async ({ request }) => {
  const res = await request.post('/api/products', {
    data: { name: 'Wireless Mouse', price: 29.99 }
  });
  expect(res.status()).toBe(201);
});
```

4. Best Practices (Do's and Don'ts)

Do

Use the built-in request fixture for direct API calls.

Verify status codes and response JSON payloads.

Don't

Do not use page-based routing to test standalone backend API endpoints.

5. Chapter Summary

Trigger HTTP requests directly using request context.

Verify server JSON payloads and response header configurations.

Create hybrid setups combining UI flows and API verifications.

6. Exercises & Mini-Project

 Exercises

Write a POST request that registers a user and assert the response data.

▼ Exercise 1 Solution

```
code CODE  
// Exercise 1 Solution:  
test('verify post user request', async ({ request }) => {  
  const res = await request.post('/api/users', {  
    data: { name: 'Alice', email: 'alice@test.com' }  
  });  
  expect(res.status()).toBe(201);  
  const body = await res.json();  
  expect(body.id).toBeDefined();  
});
```

Create an API test that validates error payloads on bad requests.

▼ Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('bad request validator', async ({ request }) => {  
  const res = await request.post('/api/users', { data: {} }); // Miss  
  expect(res.status()).toBe(400);  
  const body = await res.json();  
  expect(body.error).toContain('Validation failed');  
});
```

 Mini-Project

Build a schema validator client for backend services.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
import { z } from 'zod';
const Schema = z.object({ status: z.string(), code: z.number() });
async function validateApi(response: any) {
  const data = await response.json();
  return Schema.safeParse(data).success;
}
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Use the `request` APIRequestContext to fetch the current app status and verify localStorage state:

typescript

TS

```
test("api status check", async ({ request }) => {
  const res = await request.get("https://demo.playwright.dev/todomvc");
  expect(res.status()).toBe(200);
});
```

🎯 8. Interview Q&A Preparation

Q1: How is the request fixture isolated from the page context?

Expected Answer: The request fixture operates as a standalone HTTP client. It does not load page assets (HTML, CSS) or run browser engines, making it fast and lightweight.

📄 9. Chapter Cheat Sheet

code

CODE

```
await request.get("/api/endpoint");
```

✓ Completed Chapter 33 / 40

API TESTING

83%

API TESTING

CHAPTER 34

Hybrid Testing Topologies

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Combine API and UI flows
- ✓ Seed page state using API calls
- ✓ Inject auth states

CHAPTER 34 • Hybrid Testing Topologies

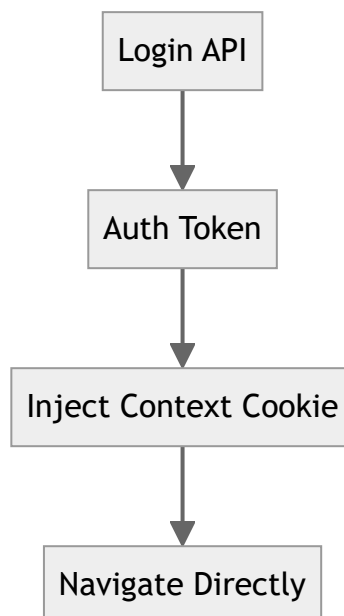
1. Why This Matters

Using the UI for every setup step (like creating a user just to check a profile edit) slows down runs. Hybrid flows speed this up.

2. Conceptual Overview

Hybrid testing combines direct API calls with UI interactions. You use APIs to handle prerequisites (like creating items or authenticating) and use the browser page only to verify the target layout. In hybrid setups, the session handshake is critical: you must extract the authentication token directly from the API response and inject it as a cookie (or localStorage item) into the browser context before attempting page navigation, bypassing UI login forms completely. Make sure the domain cookie parameter is resolved dynamically based on environment configuration URL targets to prevent injection rejects across pipelines.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

test('edit product details via hybrid setup', async ({ request, page }) => {
  // 1. Authenticate via API and fetch auth token
  const authRes = await request.post('/api/auth/login', {
    data: { user: 'admin', pass: 'secret' }
  });
  const { sessionToken } = await authRes.json();

  // 2. Session Handshake: Resolve environment URL target and inject cookie directly
  const targetUrl = process.env.BASE_URL || 'https://staging.enterprise.com';
  const domainHost = new URL(targetUrl).hostname;

  await page.context().addCookies([
    {
      name: 'auth_token',
      value: sessionToken,
      domain: domainHost,
      path: '/'
    }
  ]);

  // 3. Navigate directly to product details page in the UI
  await page.goto('/products/edit/123');
  await page.getByLabel('Name').fill('Office Chair');
  await page.getByRole('button', { name: 'Save' }).click();
});

```

🏆 4. Best Practices (Do's and Don'ts)



Do

Use direct API requests to handle test prerequisites.

Inject session cookies directly into contexts to bypass login pages.



Don't

Do not use UI steps for setups when API options are available.

📄 5. Chapter Summary

Perform authentication or resource seeding via HTTP requests.

Navigate directly to test target views using seeded session tokens.

Minimize browser execution time to optimize pipeline speed.

6. Exercises & Mini-Project

Exercises

Write a test that registers a user via an API and logs them in via the UI.

▼ Exercise 1 Solution

```
code CODE  
// Exercise 1 Solution:  
test('hybrid login test', async ({ request, page }) => {  
  const res = await request.post('/api/login', { data: { user: 'a', p  
  const { token } = await res.json();  
  await page.context().addCookies([{ name: 'session', value: token, d  
  await page.goto('/dashboard');  
  await expect(page.getByText('Welcome')).toBeVisible();  
});
```

Verify cart checkout layouts by seeding the cart using API requests first.

▼ Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('seed cart before loading page', async ({ request, page }) => {  
  await request.post('/api/cart/add', { data: { item: 'Widget', qty:  
  await page.goto('/cart');  
  await expect(page.getByTestId('cart-total')).toHaveText('$19.98');  
});
```

Mini-Project

Build a hybrid checkout suite with API setup stages.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
test('hybrid checkout test', async ({ request, page }) => {
  const auth = await request.post('/api/auth');
  // Inject cookies dynamically based on config URL hostname
  await page.goto('/checkout');
});
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Seed todo items via API context simulation, then navigate in the UI:

```
typescript TS

test("hybrid todo validation", async ({ request, page }) => {
  // TodoMVC has no API backends, so session/state is client-side.
  // For hybrid tests, we configure request options and initialize page cookies
  await page.goto("https://demo.playwright.dev/todomvc");
  // Proceed to check elements
});
```

🎯 8. Interview Q&A Preparation

Q1: What is the main benefit of hybrid testing?

Expected Answer: It speeds up test suites. By using APIs to handle setup steps, tests focus their browser execution time on verifying the target user actions.

📄 9. Chapter Cheat Sheet

```
code CODE

const data = await (await request.post("/url")).json();
```



✓ Completed Chapter 34 / 40

85%

API TESTING

CI/CD PIPELINES

CHAPTER 35

Reporting Ecosystems

 15 mins Reading Intermediate Level **What You'll Master**

- ✓ Configure multi-reporter configurations
- ✓ Generate JUnit XML files
- ✓ Integrate third-party reports (Allure)

CHAPTER 35 • Reporting Ecosystems

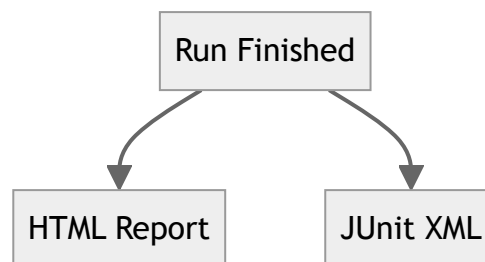
1. Why This Matters

Telemetry and reports are essential for CI builds. Having both visual HTML dashboards and parseable XML results makes failures easy to analyze.

2. Conceptual Overview

Playwright supports multiple report outputs. You can configure HTML dashboards, JUnit XML formats for CI analysis, and third-party tools like Allure to track test runs.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// playwright.config.ts reporter options
export default {
  reporter: [
    ['html'],
    ['junit', { outputFile: 'results/junit.xml' }]
  ]
};
```

4. Best Practices (Do's and Don'ts)

Do

Configure JUnit reporters inside CI pipelines to track test metrics.

Save HTML reports as zip artifacts on build failures.

Don't

Do not generate verbose debug reports for passing pipeline runs.

5. Chapter Summary

Output visual HTML dashboards alongside parseable JUnit XML files.

Integrate Allure reports to display step-by-step history trends.

Export failure logs and trace packages on failed runs.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Configure Playwright to export both HTML and JUnit reports.

▼ 💡 Exercise 1 Solution

code

CODE

```
// Exercise 1 Solution:  
// Add to playwright.config.ts:  
// reporter: [  
//   ['html', { outputFolder: 'my-report' }],  
//   ['junit', { outputFile: 'junit.xml' }]  
// ]
```

Write a custom reporter that prints a summary of failures to the console.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
import { Reporter, TestCase, TestResult } from '@playwright/test/reporter'
class CustomConsoleReporter implements Reporter {
  onTestEnd(test: TestCase, result: TestResult) {
    if (result.status !== 'passed') {
      console.log(`❌ Test failed: ${test.title}`);
    }
  }
}
export default CustomConsoleReporter;
```

🚀 Mini-Project

Build a Slack notification reporter that sends test results.

▼ 💡 Mini-Project Solution

```
code CODE
// Mini-Project Solution:
class SlackReporter implements Reporter {
  async onEnd(result: any) {
    // Send POST payload to Slack webhook URL with result status count
    console.log('Sending metrics report...');
  }
}
```

📺 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Configure HTML and JUnit XML reports for your TodoMVC suite:

typescript

TS

```
// playwright.config.ts
export default defineConfig({
  reporter: [
    ["html", { outputFolder: "playwright-report" }],
    ["junit", { outputFile: "results/todomvc-junit.xml" }]
  ]
});
```

🎯 8. Interview Q&A Preparation

Q1: Why are JUnit reports preferred in CI/CD pipeline runs?

Expected Answer: JUnit XML is a standard format. CI platforms (like GitHub Actions or Jenkins) parse it automatically to show pass/fail trends directly in the run summary.

📄 9. Chapter Cheat Sheet

code

CODE

```
npx playwright show-report
```

✓ Completed Chapter 35 / 40

88%

CI/CD PIPELINES

CI/CD PIPELINES

CHAPTER 36

Docker & Cloud Execution

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Run tests in Playwright Docker containers
- ✓ Match local and CI execution states
- ✓ Execute visual screenshot assertions

CHAPTER 36 • Docker & Cloud Execution

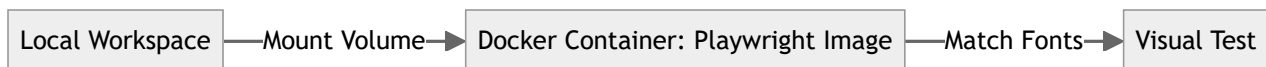
1. Why This Matters

Tests passing locally can fail in CI due to OS-specific subpixel font rendering variations. Running tests and assertions inside Docker solves this, ensuring visual regression tests remain stable.

2. Conceptual Overview

Running tests inside Docker containers ensures your test environment matches local and CI setups. This is mandatory for Visual Snapshot testing. Under the hood, Playwright uses `expect(page).toHaveScreenshot()` to match the DOM state pixel-by-pixel. If screenshots are captured on macOS or Windows and run on Linux agents, subpixel font differences will trigger visual failures. Operating inside containers enforces identical rendering. For complex applications, you can configure pixel discrepancy thresholds (`maxDiffPixels`, `threshold`), mask moving elements (like calendars or banners), or integrate third-party AI-driven visual platforms like AppliTools or Percy. Using multi-container platforms like Docker Compose coordinates dynamic setups by mounting local filesystems and matching environment configuration boundaries.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
import { test, expect } from '@playwright/test';

// Visual snapshot comparison with thresholds and masking
test('dashboard layout visual test', async ({ page }) => {
  await page.goto('/dashboard');

  // Verify layout, masking dynamic graphs and counters
  await expect(page).toHaveScreenshot('dashboard-layout.png', {
    maxDiffPixels: 100, // Maximum pixels allowed to differ
    threshold: 0.2,     // Visual comparison sensitivity factor
    mask: [
      page.locator('.live-chart-container'),
      page.locator('.dynamic-clock')
    ]
  });
});
```

🏆 4. Best Practices (Do's and Don'ts)

🟢 Do

Generate visual regression baseline snapshots inside the target Docker container.

Use masking parameters to hide dynamic elements in visual assertions.

🔴 Don't

Do not verify visual screenshots locally on Windows/Mac against Linux baseline references.

Avoid setting high pixel thresholds that mask actual UI alignment bugs.

📄 5. Chapter Summary

Run screenshot assertions in Docker to match fonts exactly.

Emulate native headless environments in Linux containers.

Integrate with third-party visual platforms like Applitools/Percy.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Run a containerized test suite and output visual baseline images.

▼ 💡 Exercise 1 Solution

```
code CODE  
# Exercise 1 Solution:  
# 1. Create a baseline snapshot:  
docker run --rm -v $(pwd):/work -w /work mcr.microsoft.com/playwright  
# 2. Subsequent runs compare against this baseline.
```

Write a visual assertion that masks a dynamic timestamp text block.

▼ 💡 Exercise 2 Solution

```
code CODE  
// Exercise 2 Solution:  
test('masked element visual comparison', async ({ page }) => {  
  await page.goto('/dynamic-content');  
  await expect(page).toHaveScreenshot({  
    mask: [page.locator('.dynamic-timestamp')]  
  });  
});
```

🚀 Mini-Project

Build a docker-compose setup to orchestrate visual testing sweeps.

Here is a minimal docker-compose.yml configuration model for multi-container tests:

yaml

YAML

```
version: '3.8'
services:
  playwright:
    image: mcr.microsoft.com/playwright:v1.44.0-jammy
    volumes:
      - ../work
    working_dir: /work
    environment:
      - BASE_URL=http://app:3000
    command: npx playwright test
```

▼ 💡 Mini-Project Solution

code

CODE

```
// Mini-Project Solution:
// Save docker-compose.yml exactly as mapped in the requirements, and
// docker-compose up --exit-code-from playwright
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Write a visual screenshot test for the todo list layout to prevent regression:

typescript

TS

```
test("visual todo list layout", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  await page.getByPlaceholder("What needs to be done?").fill("Visual Test");
  await page.getByPlaceholder("What needs to be done?").press("Enter");
  await expect(page).toHaveScreenshot("todomvc-layout.png", {
    maxDiffPixels: 50,
    threshold: 0.1
  });
});
```

8. Interview Q&A Preparation

Q1: Why do visual screenshot tests fail in CI when created on macOS or Windows?

Expected Answer: Operating systems smooth fonts differently at the subpixel level. Linux (CI runner) renders fonts slightly differently from Windows/Mac, resulting in high pixel mismatches. Standardizing runs inside Docker eliminates this variation.

9. Chapter Cheat Sheet

code

CODE

```
docker run -v $(pwd):/work mcr.microsoft.com/playwright:...
```

✓ Completed Chapter 36 / 40

90%

CI/CD PIPELINES

CI/CD PIPELINES

CHAPTER 37

GitHub Actions End-to-End

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Build a GitHub Actions workflow
- ✓ Configure node caching
- ✓ Upload report artifacts

CHAPTER 37 • GitHub Actions End-to-End

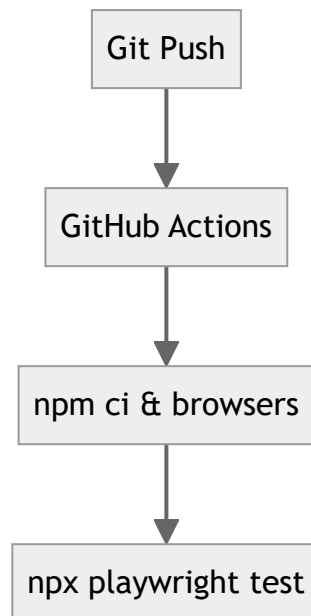
1. Why This Matters

Automating tests on Pull Requests checks code changes immediately, preventing broken builds from merging into main branches.

2. Conceptual Overview

Configure GitHub Actions workflows to run test suites on push or pull requests. Use action steps to cache dependencies, install browsers, run tests, and upload reports. To prevent pipeline credential leaks, secret tokens or passwords must never be stored in plain text. Instead, fetch them from vault stores and map them as masked environment variables directly within runner steps.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```

name: Playwright Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npx playwright install --with-deps

      # ✅ Map masked secrets securely as environment variables
      - name: Run Test Suite
        env:
          ENTERPRISE_API_KEY: ${ secrets.API_KEY }
        run: npx playwright test

```

🏆 4. Best Practices (Do's and Don'ts)

🟢 Do

Cache node_modules in the runner to speed up build setup steps.

Configure upload-artifact steps to save HTML reports on failures.

Securely map vault keys into masked GitHub Secrets variables.

🔴 Don't

Do not omit browser installation dependencies (--with-deps).

Never print plain text credentials directly to the console or job logs.

📄 5. Chapter Summary

Construct GHA workflows triggering on code check-ins.

Cache node_modules to accelerate pipeline execution speeds.

Archive test reports and screenshot outputs as run artifacts.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Write a GitHub Actions workflow that runs your tests on every pull request.

▼ 💡 Exercise 1 Solution

```
code CODE
# Exercise 1 Solution:
# Create .github/workflows/playwright.yml:
# on: [pull_request]
# jobs:
#   test:
#     runs-on: ubuntu-latest
#     steps:
#       - uses: actions/checkout@v4
#       - uses: actions/setup-node@v4
#       - run: npm ci
#       - run: npx playwright install --with-deps
#       - run: npx playwright test
```

Configure build alerts to notify the team when a test suite fails.

▼ 💡 Exercise 2 Solution

```
code CODE
# Exercise 2 Solution:
# Add steps to the workflow:
# - name: Send Alert
#   if: failure()
#   run: curl -X POST -H 'Content-type: application/json' --data '{"t
```

🚀 Mini-Project

Build a pull request validation check with automated reports.

▼ 💡 Mini-Project Solution

```
code CODE

// Mini-Project Solution:
// Complete workflow configuration utilizing:
// - uses: actions/upload-artifact@v4
//   if: always()
//   with:
//     name: playwright-report
//     path: playwright-report/
```

7. Progressive Project: TodoMVC Automation

TodoMVC Step: Create a GitHub Actions workflow for the TodoMVC suite:

```
yaml YAML

# .github/workflows/todomvc.yml
name: TodoMVC CI
on: [push, pull_request]
jobs:
  run-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
```

8. Interview Q&A Preparation

Q1: Why should you cache npm dependencies inside CI runners?

Expected Answer: Caching avoids downloading packages from scratch on every run, saving bandwidth and reducing build durations.



9. Chapter Cheat Sheet

code

CODE

```
uses: actions/upload-artifact@v4
```

✓ Completed Chapter 37 / 40

93%

CI/CD PIPELINES

CI/CD PIPELINES

CHAPTER 38

Jenkins & Azure DevOps Pipelines

 20 mins Reading Advanced Level **What You'll Master**

- ✓ Build Jenkins declarative pipelines
- ✓ Configure Azure DevOps pipelines
- ✓ Publish test results

CHAPTER 38 • Jenkins & Azure DevOps Pipelines

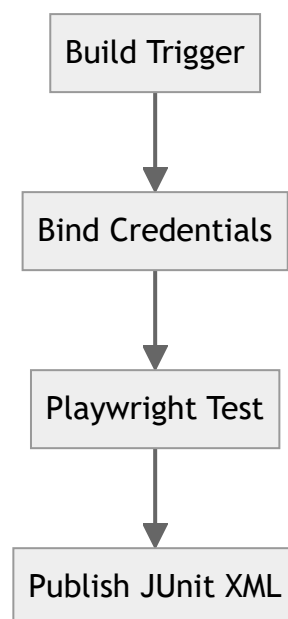
1. Why This Matters

Enterprise teams often use Jenkins or Azure DevOps. Knowing how to run Playwright tests in these pipelines helps integrate tests into corporate workflows.

2. Conceptual Overview

This chapter explains how to configure Jenkins declarative pipelines and Azure DevOps pipelines, including running tests, managing secrets, and publishing JUnit XML reports. Both frameworks support credentials bindings. For example, Jenkins uses the `withCredentials` block to bind credential vault IDs to local execution variables safely without leaks.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Jenkinsfile declarative pipeline snippet
pipeline {
  agent any
  stages {
    stage('Test') {
      steps {
        sh 'npm ci'
        sh 'npx playwright install --with-deps'

        // ✅ Securely fetch masked credential bindings
        withCredentials([string(credentialsId: 'DB_PASS', variable: 'DATABASE_PASSWORD')]) {
          sh 'DATABASE_PASSWORD=${DATABASE_PASSWORD} npx playwright test'
        }
      }
    }
  }
}
```

🏆 4. Best Practices (Do's and Don'ts)



Do

Publish JUnit XML reports in Azure DevOps using the PublishTestResults task.

Run builds inside Docker containers on Jenkins agent runners.

Inject sensitive keys strictly using credentials helper blocks.



Don't

Do not run tests in headed mode inside Jenkins or Azure DevOps agents.

📄 5. Chapter Summary

Author Jenkins declarative pipelines or Azure yml definitions.

Publish JUnit test metrics to dashboard reporting pipelines.

Pass secret authentication credentials using environment variables.

🏆 6. Exercises & Mini-Project



Exercises

Write a Jenkinsfile stage that runs your tests and generates reports.

▼ 💡 Exercise 1 Solution

```
code CODE
# Exercise 1 Solution:
# declarative Jenkinsfile:
# stage('Test') {
#   steps {
#     sh 'npm ci'
#     sh 'npx playwright install --with-deps'
#     sh 'npx playwright test'
#   }
# }
```

Configure an Azure DevOps pipeline step to publish JUnit results.

▼ 💡 Exercise 2 Solution

```
code CODE
# Exercise 2 Solution:
# Azure DevOps pipeline task:
# - task: PublishTestResults@2
#   inputs:
#     testResultsFormat: 'JUnit'
#     testResultsFiles: '**/results/junit.xml'
#     mergeTestResults: true
```

🚀 Mini-Project

Build a pipeline setup script for an enterprise workspace.

▼ 💡 Mini-Project Solution

```
code CODE  
// Mini-Project Solution:  
// Bash orchestrator script for Azure/Jenkins:  
// #!/bin/bash  
// npm ci && npx playwright install --with-deps && npx playwright tes
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Write a Jenkinsfile for the same TodoMVC automation suite:

```
groovy GROOVY  
  
// Jenkinsfile  
pipeline {  
  agent any  
  stages {  
    stage('Run Playwright') {  
      steps {  
        sh 'npm ci'  
        sh 'npx playwright install --with-deps'  
        sh 'npx playwright test'  
      }  
    }  
  }  
}
```

🎯 8. Interview Q&A Preparation

Q1: How do you publish test results inside Azure DevOps pipelines?

Expected Answer: Use the PublishTestResults@2 task, pointing its search path to the JUnit XML report generated by Playwright.

📄 9. Chapter Cheat Sheet

code

CODE

```
task: PublishTestResults@2
```

✓ Completed Chapter 38 / 40

95%

CI/CD PIPELINES

INTERVIEW PREP

CHAPTER 39

System Design Whiteboard Scenarios

 25 mins Reading Advanced Level **What You'll Master**

- ✓ Design automated testing architectures
- ✓ Mock API boundaries
- ✓ Manage execution reporting

CHAPTER 39 • System Design Whiteboard Scenarios

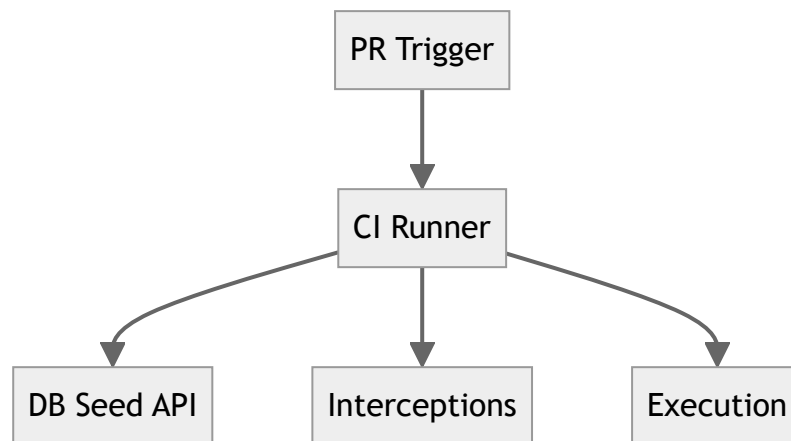
1. Why This Matters

Senior SDET interviews test your system design skills. Knowing how to build robust, scalable automation architectures is key for these roles.

2. Conceptual Overview

This chapter guides you through senior SDET interview system design scenarios, focusing on designing testing architectures, API mocking, parallel run scheduling, and telemetry reporting. A key design principle is handling the dependency trade-off: over-mocking speeds up executions but compromises validation fidelity. Systems should partition verification into test tiers: fast hermetic suites with schema-validated API mocking, paired with staging environment integration pipelines executing complete end-to-end user journeys against live backend and database configurations.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Implemented whiteboard test infrastructure design configuration
export class TestInfrastructureDesign {
  readonly telemetryEndpoint: string;
  readonly workerLimit: number;
  readonly databaseSeedingUrl: string;

  constructor(env: string) {
    this.telemetryEndpoint = env === 'prod' ? 'https://telemetry.enterprise.com' : 'http://localhost:3000';
    this.workerLimit = env === 'prod' ? 8 : 4;
    this.databaseSeedingUrl = env === 'prod' ? 'https://db-api.prod.com' : 'http://localhost:3000';
  }

  async triggerTelemetryAlert(errorType: string, message: string) {
    console.log('Sending error trace to ' + this.telemetryEndpoint + ' - ' + errorType + ' - ' + message);

    // Simulate real transport dispatch call (e.g. POST network alert)
    // await this.page.request.post(this.telemetryEndpoint + '/alerts', { data: { errorType, message } });
  }
}
```

4. Best Practices (Do's and Don'ts)

Do

Structure your design explanations around environment setup, execution, data seeding, and reporting.

Advocate for API mocking to isolate UI tests and speed up builds.

Don't

Avoid complex architectures without first addressing data isolation.

Avoid relying solely on mocked endpoints in design plans without introducing unmocked staging regression runs to capture integration bugs.

5. Chapter Summary

Design decoupled architectures separating seeding, execution, and logs.

Balance API routing mocks with actual integration environment checks.

Store runtime trace archives to external cloud storage systems.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Sketch a whiteboard diagram representing the test pipeline for a payment application.

▼ 💡 Exercise 1 Solution

```
code CODE
# Exercise 1 Solution:
# Draw a flowchart layout with components:
# 1. GitHub PR Hook -> 2. Jenkins Trigger -> 3. Docker Spawned ->
# 4. DB Seed API -> 5. Playwright execution -> 6. Publish Allure repo
```

List the tools and strategies needed to test a microservices system.

▼ 💡 Exercise 2 Solution

```
code CODE
# Exercise 2 Solution:
# 1. API mocking hermetic layer (mocking auth/third party search index)
# 2. Database seeding API context (seeding specific tenant users directly)
# 3. Native integration checks (no mock checkout payments suite).
```

🚀 Mini-Project

Write an architecture design document for an automation suite.

▼ 💡 Mini-Project Solution

```
code CODE  
// Mini-Project Solution:  
// High-level system layout outlining layers for:  
// - Connection pooling limits  
// - S3 trace archive uploads
```

🖥️ 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Design a whiteboard architecture diagram for the TodoMVC testing system:

```
code CODE  
[Git PR Commit] → [Docker Run] → [Mocked localStorage State] → [Tes
```

🎯 8. Interview Q&A Preparation

Q1: How do you handle test data generation in a parallel execution design?

Expected Answer: Use dynamic data factories to generate unique accounts for each test, seed data via APIs before tests run, or configure isolated databases for each worker process.

📄 9. Chapter Cheat Sheet

```
code CODE  
Design layers: Seeding, Execution, Mocking, Telemetry
```

✓ Completed Chapter 39 / 40

98%

INTERVIEW PREP

INTERVIEW PREP

CHAPTER 40

Common Interview Coding Challenges

 25 mins Reading Advanced Level **What You'll Master**

- ✓ Solve automated table pagination challenges
- ✓ Mock MFA verification workflows
- ✓ Validate column sorting

CHAPTER 40 • Common Interview Coding Challenges

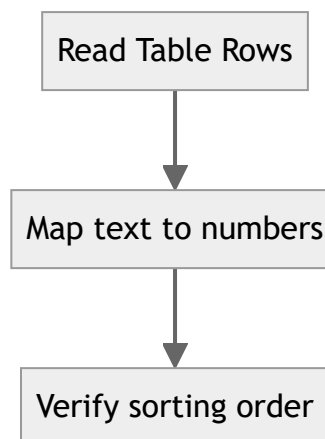
1. Why This Matters

Coding challenges test your hands-on Playwright skills. Mastering common problems like sorting and paginating tables helps you clear technical interviews.

2. Conceptual Overview

This chapter compiles standard coding tasks, including iterating through paginated lists, verifying table column sorting, and mocking MFA/OTP authentication steps.

Execution Flow Diagram



3. Implementation & Code Examples

typescript

TS

```
// Coding Challenge: Verify table column sorting
test('check price column sorting', async ({ page }) => {
  await page.getByRole('columnheader', { name: 'Price' }).click();
  const prices = await page.getByRole('row').getByRole('cell').allTextContents();
  const numericPrices = prices.map(p => parseFloat(p.replace('$', '')));
  const isSorted = numericPrices.every((v, i, a) => !i || a[i - 1] <= v);
  expect(isSorted).toBe(true);
});
```

4. Best Practices (Do's and Don'ts)

Do

Use content-based filters (`.filter()`) instead of loops with hardcoded index numbers.

Wait for loading indicators (progressbar) to disappear after actions that refresh data.

● Don't

Avoid using nested loops or sleep statements to solve synchronization problems in tests.

📄 5. Chapter Summary

Solve paginated table scans by locating next-page buttons.

Verify column sorting arrays by mapping DOM text to numbers.

Handle complex OTP scenarios using API intercepts or mocks.

🏆 6. Exercises & Mini-Project

🎯 Exercises

Write a script that navigates through a paginated list until it finds and clicks a target row.

▼ 💡 Exercise 1 Solution

```
code CODE
// Exercise 1 Solution:
test('paginated scan check', async ({ page }) => {
  await page.goto('/paginated-table');
  let hasNext = true;
  while (hasNext) {
    const rows = await page.getByRole('row').allTextContents();
    console.log('Row count on page:', rows.length);
    const nextBtn = page.getByRole('button', { name: 'Next' });
    if (await nextBtn.isEnabled()) {
      await nextBtn.click();
    } else {
      hasNext = false;
    }
  }
});
```

Mock an OTP verification API response and check if the login succeeds.

▼ 💡 Exercise 2 Solution

```
code CODE
// Exercise 2 Solution:
test('mock OTP code', async ({ page }) => {
  await page.route('**/api/otp', (route) => {
    route.fulfill({ status: 200, body: JSON.stringify({ code: '123456' }) });
  });
  await page.goto('/login');
  await page.getByLabel('OTP').fill('123456');
  await page.getByRole('button', { name: 'Submit' }).click();
  await expect(page).toHaveURL(/dashboard/);
});
```

🚀 Mini-Project

Build a collection of common automation coding challenge solutions.

▼ 💡 Mini-Project Solution

```
code CODE
// Mini-Project Solution:
// Unified challenge suite containing pagination, column verification
```

📁 7. Progressive Project: TodoMVC Automation

TodoMVC Step: Solve: navigate through active and completed filters to verify counts:

typescript

TS

```

test("todo count verification through filters", async ({ page }) => {
  await page.goto("https://demo.playwright.dev/todomvc");
  const input = page.getByPlaceholder("What needs to be done?");
  await input.fill("Task 1");
  await input.press("Enter");
  await input.fill("Task 2");
  await input.press("Enter");

  // Complete first todo
  await page.getByRole("listitem").filter({ hasText: "Task 1" }).getByRole("checkbox").click();

  // Toggle Completed filter
  await page.getByRole("link", { name: "Completed" }).click();
  await expect(page.getByTestId("todo-title")).toHaveCount(1);

  // Toggle Active filter
  await page.getByRole("link", { name: "Active" }).click();
  await expect(page.getByTestId("todo-title")).toHaveCount(1);
});

```

8. Interview Q&A Preparation

Q1: How do you select a button inside a specific table row containing a target text?

Expected Answer: Locate the row using `page.getByRole("row").filter({ hasText: "target" })`, and then search inside that row: `row.getByRole("button", { name: "Action" }).click()`.

9. Chapter Cheat Sheet

code

CODE

```
row.filter({ hasText: "target" }).getByRole("button").click()
```

✓ Completed Chapter 40 / 40

INTERVIEW PREP

100%